

Sistemi informativi su Web

HTML e CSS

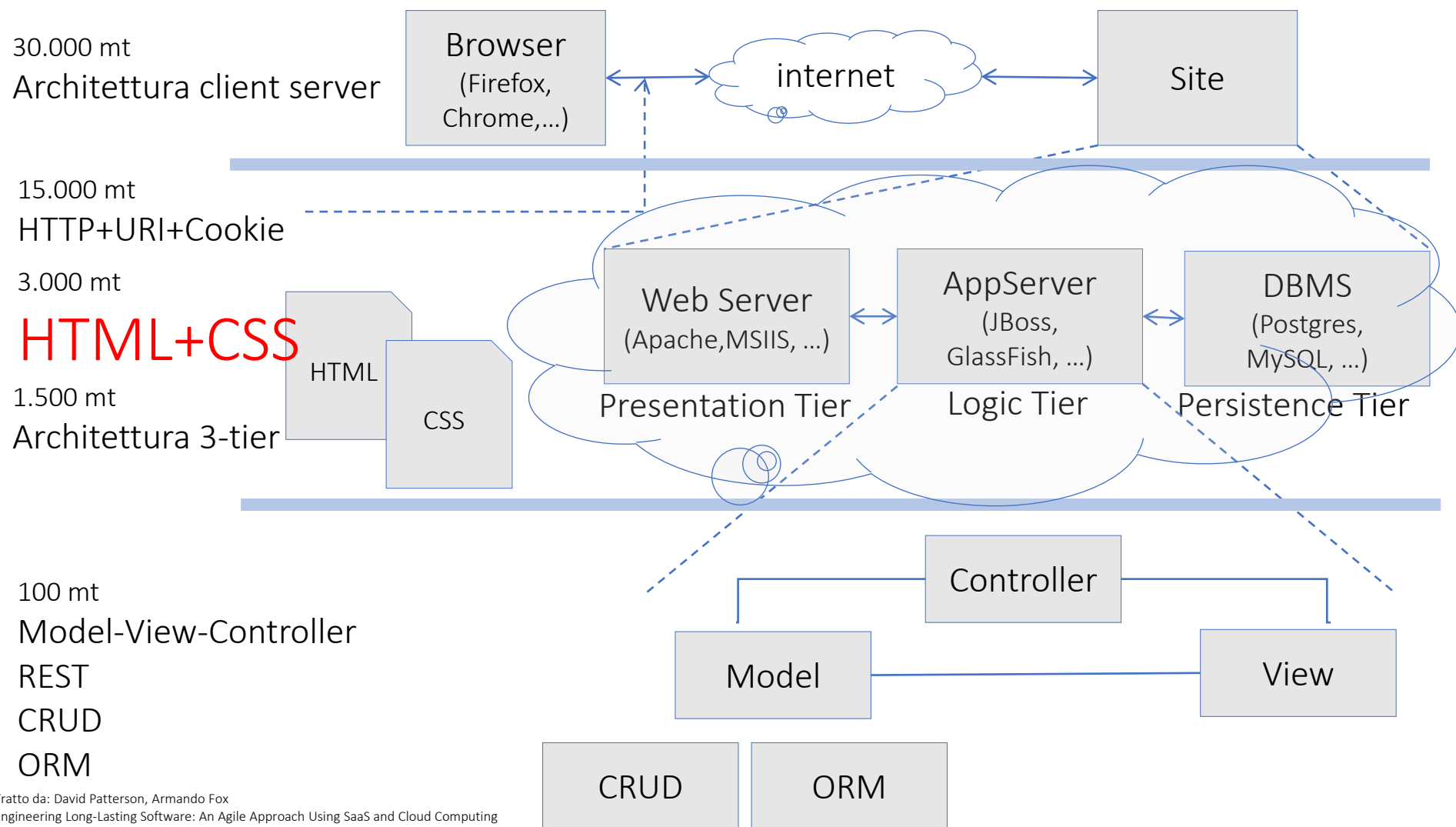
aa 2024-2025



Paolo Merialdo
Università degli Studi Roma Tre

CREDITS: these slides are based on original material by Victoria Kirst

Anatomia di un sistema informativo su Web



HTML

HTML (Hypertext Markup Language)

- Descrive il **contenuto** e la **struttura** di una pagina Web composta di blocchi chiamati **elementi**.

<p>

HTML è divertente!!!

</p>

Selettori CSS:

Classi e identificatori

Classi e identificatori

Ci sono 3 tipi base di selettori CSS:

Elemento (usato fino ad ora)	p	Tutti gli elementi <p>
ID selector	#abc	elementi con id="abc"
Class selector	.abc	elementi con class="abc"

```
<h1 id="title">Homework</h1>
```

```
<em class="hw">HW0</em> da consegnare Venerdì.<br/>
```

```
<strong class="hw">HW1</strong> data da definire.<br/>
```

```
<em>Tutti gli homework vanno consegnati entro le 22.00.</em>
```

Altri selettori

element.className

Sintassi	Esempio	Descrizione
<code>element.className</code>	<code>p.abc</code>	elementi <code><p></code> con classe <code>abc</code>

HTML

```
<h1 class="hw">Homework 0</h1>
<p class="hw">Due Fri</p>
<p class="hw">Late cutoff Sun</p>
<h1>Lectures</h1>
<p>Apr 3: Syllabus</p>
<p>Apr 5: HTML+CSS</p>
```

CSS

```
p.hw {
  color: green;
}
```

Homework 0

Due Fri

Late cutoff Sun

Lectures

Apr 3: Syllabus

Apr 5: HTML+CSS

Descendent selector

Sintassi	Esempio	Descrizione
selettore selettore	div strong	Con riferimento all'albero associato al documento HTML, elementi che sono discendenti di <div>

HTML

```
<div class="hw">
  <h1>Homework 0</h1>
  <p>Due Fri</p>
  <p>Late cutoff Sun</p>
</div>
```

CSS

```
.hw p {
  color: green;
}
```

Homework 0

Due Fri

Late cutoff Sun

Lectures

Apr 3: Syllabus

Apr 5: HTML+CSS

Descendent selector

sintassi	Esempio	Descrizione
selettore selettore	div strong	Con riferimento all'albero associato al documento HTML, elementi che sono discendenti di <div>

Nota: l'elemento è un discendente, non strettamente il figlio

HTML

```
<div class="hw">
  <div>
    <p>
      HW0: <strong>Due Friday</strong>
    </p>
  </div>
  HW1 out <strong>Monday</strong>
</div>
```

CSS

```
.hw strong {
  color: red;
}
```

HW0: **Due Friday**
HW1 out **Monday**

Child selector

sintassi	Esempio	Descrizione
selettore > selettore	div > strong	Con riferimento all'albero associato al documento HTML, elementi che sono figli diretti di <div>

selector, selector (virgola)

Sintassi	Esempio	Descrizione
selector, selector	h2, div	elementi <h2> e <div>

 **HTML** 

```
<h1>Course Info</h1>
<h2>Lectures</h2>
<p>Mon-Wed-Fri 1:30-2:20</p>
<h2>Honor code</h2>
<p>Do the right thing</p>
```

 **CSS** 

```
h1, h2 {
  font-family: Arial;
}
```

Course Info

Lectures

Mon-Wed-Fri 1:30-2:20

Honor code

Do the right thing

Riassunto

Esempio	Descrizione
p	Tutti gli elementi <p>
.abc	Tutti gli elementi con classe abc , cioè class="abc"
#abc	L'elemento con id abc , cioè id="abc"
p.abc	Elementi <p> con classe abc
p#abc	Elementi <p> con id abc id (nota che p è ridondante!)
div strong	Elementi che sono discendenti di un elemento <div>
div > strong	Elementi che sono figli di un elemento <div>
h2, div	Tutti gli elementi <h2> e <div>

Combinazioni di selettori

I selettori possono essere combinati:

```
#main li.important strong {  
  color: red;  
}
```

Q: quali riquadri seleziona?

Combinazioni di selettori

I selettori possono essere combinati:

```
#main li.important strong {  
  color: red;  
}
```

Q: quali riquadri seleziona?

Elementi **** che sono discendenti di
elementi **** che hanno una classe **"important"**
che sono discendenti dell'elemento che ha id **"main"**

Collisioni

Quando ci sono collisioni, vince la regola più specifica ([specificity](#))

```
div strong { color: red; }  
strong { color: blue; }
```

```
<div>  
    <strong>What color am I?</strong>  
</div>
```

Collisioni

Quando ci sono collisioni, vince la regola più specifica ([specificity](#))

```
div strong { color: red; }  
strong { color: blue; }
```

```
<div>  
  <strong>What color am I?</strong>  
</div>
```

Collisioni

Regole di precedenza per specificità ([dettagli](#)):

- **id** più specifico di **class**
- **class** più specifico del nome dell'elemento
- Regole che puntano direttamente ad un elemento sono più specifiche delle regole ereditate (vedi oltre)

Collisioni

- In caso di uguale specificità, vince l'ultima regola.

```
strong { color: red; }  
  strong { color: blue; }
```

```
<div>  
  <strong>What color am I?</strong>  
</div>
```

Il processo di definizione di quale regola si applica si chiama [cascade](#).
Da questa caratteristica, la "C" di CSS (Cascading Style Sheets).

Ereditarietà

Con riferimento all'albero associato al documento HTML, gli stili CSS sono ereditati da padre a figlio.

Invece di selezionare
tutti gli elementi
individualmente,

```
a, h1, p, strong {  
    font-family: Helvetica;  
}
```

possiamo impostare lo
stile su un antenato: lo
stile è ereditato dai
discendenti.

```
body {  
    font-family: Helvetica;  
}
```

Lo stile può essere
sovrascritto

```
h1, h2 {  
    font-family: Consolas;  
}
```

Ereditarietà

Non tutte le proprietà CSS sono ereditate.

```
a {  
  border: solid;  
  font-family: Arial;  
}
```

`<em>` eredita la proprietà `font-family`, ma non la proprietà `border`

```
<a href="/home">  
  Back to <em>Home</em>  
</a>
```

[Back to Home](#)

Ereditarietà

Non tutte le proprietà CSS sono ereditate.

- Quali proprietà sono ereditate?
- Ce lo dicono le specifiche.

Nel dubbio, controlliamole su MDN. Esempio:

<u>font-family</u> :	Inherited	yes
<u>border</u> :	Inherited	no

- In genere, le proprietà relative al testo sono ereditate, quelle relative alla disposizione no.

Pseudo classi CSS

[pseudo-classes](#): permettono associare uno *stato dinamico* al selettore. Le regole si applicano quando gli elementi selezionati si trovano in quello stato

Sintassi	Semantica
a	Tutti gli elemtni a (ancora di un link ipertestuale)
a:visited	L'ancora di un link visitato
a:link	L'ancora di un link non visitato
a:hover	L'ancora di un link sorvolato dal mouse
a:active	L'ancora di un link attivato dal mouse (click)

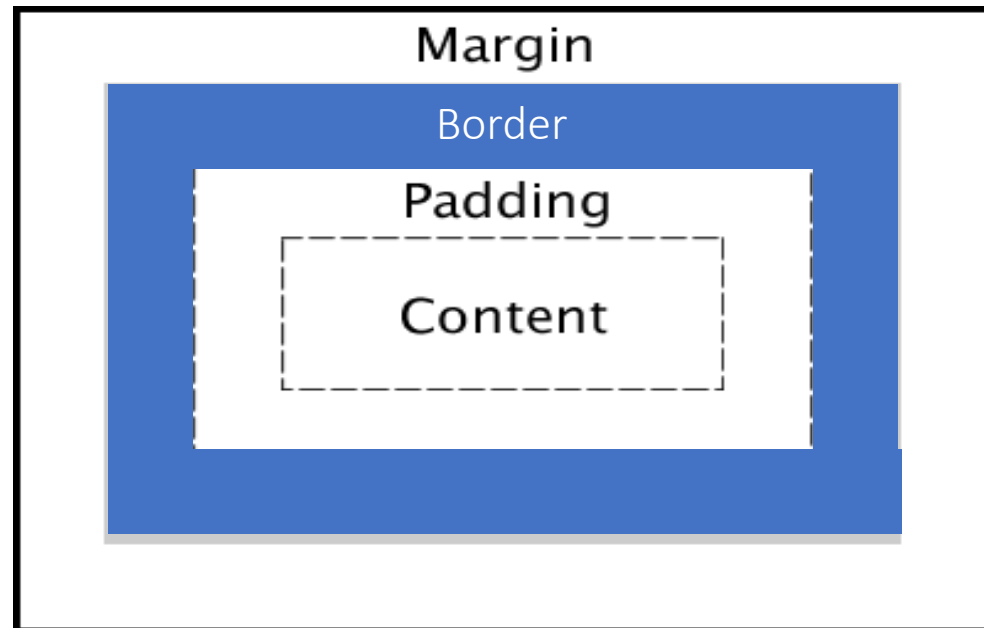
Altre [pseudo-classi](#)

CSS Box Model

CSS Box Model

Ogni elemento è composto di 4 strati:

- Il contenuto
- Il bordo (**border**) intorno al contenuto
- Lo spazio al bordo (**padding**), che separa il contenuto dal bordo
- Un margine esterno (**margin**) che separa il bordo dai riquadri degli elementi circostanti



[JsBin](#)

border

Si possono definire individualmente:

```
border-top  
border-bottom  
border-left  
border-right
```

Analogamente, vale sulle proprietà:

```
border-style: dotted;  
border-width: 3px;  
border-color: purple;
```

([tutti gli stili](#))

border

Si possono definire individualmente:

```
border-top  
border-bottom  
border-left  
border-right
```

Analogamente, vale sulle proprietà:

```
border-style: dotted;  
border-width: 3px;  
border-color: purple;
```

([tutti gli stili](#))

Vedremo che ci sono
unità di misura oltre ai
pixel

Riquadri arrotondati

La proprietà `border-radius` arrotonda gli angoli:
`border-radius: 10px;`

padding

La proprietà padding permette di definire la distanza del contenuto dal bordo

```
div {border: solid;
```

```
p { border: dashed;  
padding: 3px;  
margin: 10px}
```

Scopriamo il box model

Non è difficile

[JsBin](#)

padding: spazio tra il contenuto e il bordo

- padding-top, padding-bottom, padding-left, padding-right

- shorthand:

padding: **2px** **4px** **3px** **1px**; <- top|right|bottom|left

padding: **10px** **2px**; <- top+bottom|left+right

margin

L proprietà margin permette di definire la spaziature tra elementi contigui

```
div {border: solid;
```

```
p { border: dashed;  
padding: 3px;  
margin: 10px}
```

Scopriamo il box model

Non è difficile

[JsBin](#)

margin regola lo spazio tra elementi contigui

- Si possono specificare **margin-top**, **margin-bottom**, **margin-left**, **margin-right**
- shorthand:
margin: 2px 4px 3px 1px; <- **top**|**right**|**bottom**|**left**
margin: 10px 2px; <- **top+bottom**|**left+right**

margin

Perchè otteniamo questo:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>JS Bin</title>
</head>
<body>
  <div>
    Scopriamo il box model
  </div>
  <div>
    Scopriamo il box model
  </div>
  <p>
    Non è difficile
  </p>
</body>
</html>
```

```
div {border: solid;
      margin: 10px;
}
```

```
div { border: dashed;
      padding: 3px;
      margin: 10px;
}
```

Scopriamo il box model

Scopriamo il box model

Non è difficile

al posto di questo?

Scopriamo il box model

Scopriamo il box model

Collasso di margin

Generalmente, se:

- Gli elementi sono fratelli e block-level

Lectures

Homework

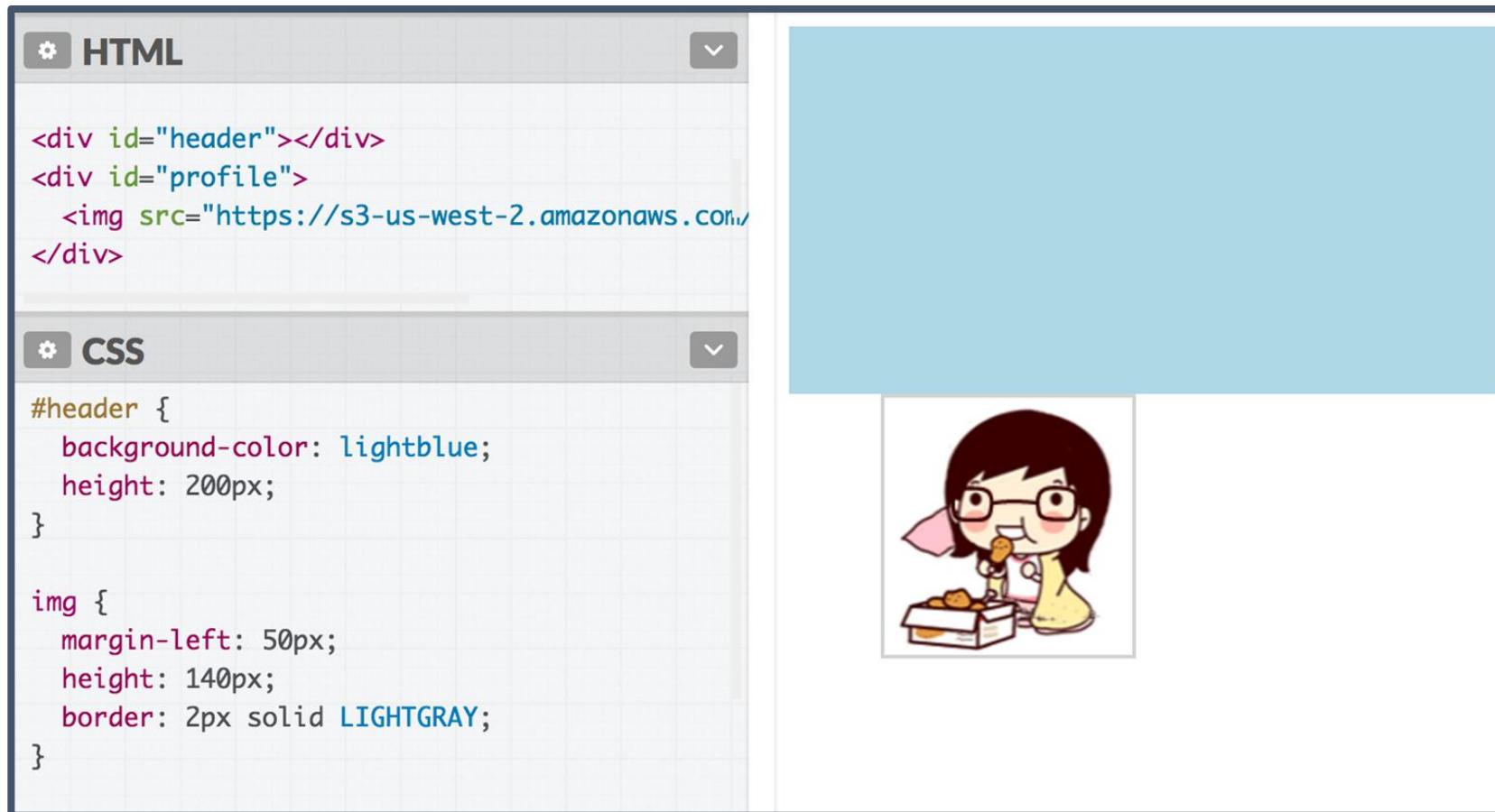
Syllabus

collassano a $\max(\text{Bottom Margin}, \text{Top Margin})$.

(Ci sono [eccezioni](#))

margin: valori negativi

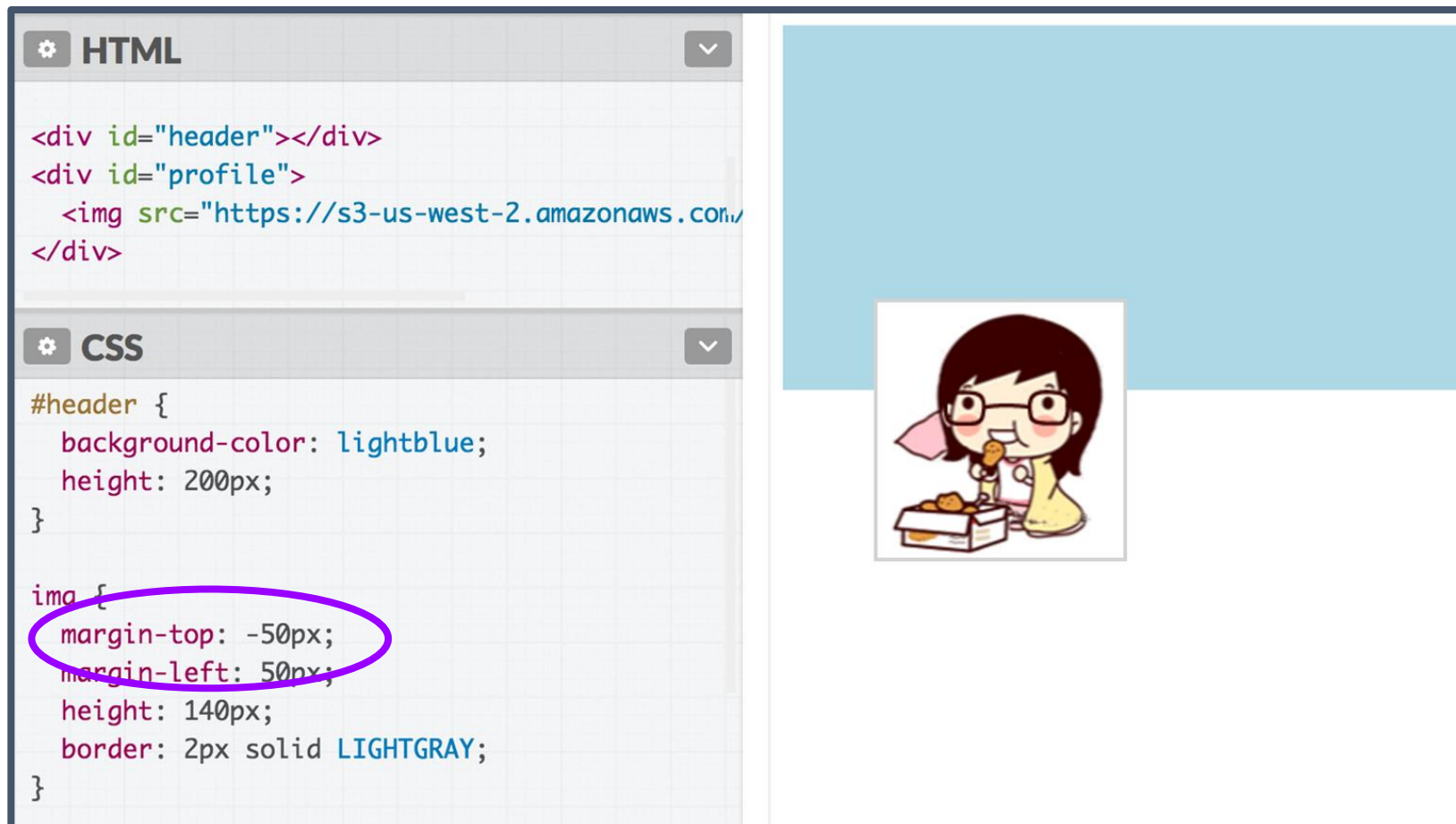
I margini possono essere negativi



margin: valori negativi

I margini possono essere negativi

- `img { margin-top: -50px; }`



Box model per elementi inline?

Q: Il box model si applica anche agli elementi inline?

Box model per elementi inline?

Q: Il box model si applica anche agli elementi inline?

A: Sì, ma ha un comportamento diverso.

Per dettagli vedi [una forma diversa](#)

Riassumiamo...

- Elementi block vs inline
- Selettori (classi e identificatori)
- div e span (ed creazione di elementi generici)
- CSS box model (content -> padding -> border -> margin)

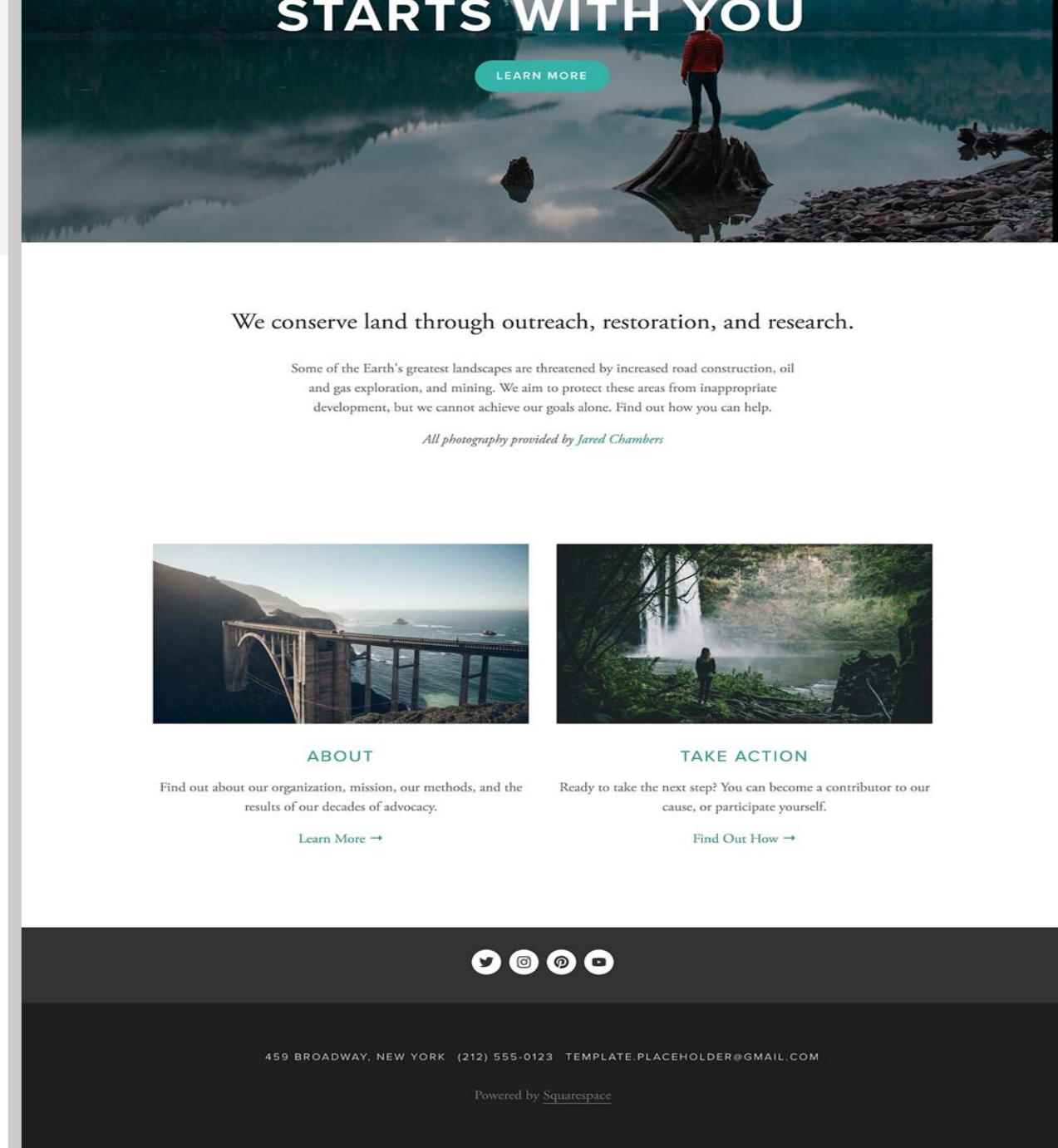
Proviamo a fare una pagina "reale"!

Layout: esercizio

Esercizio

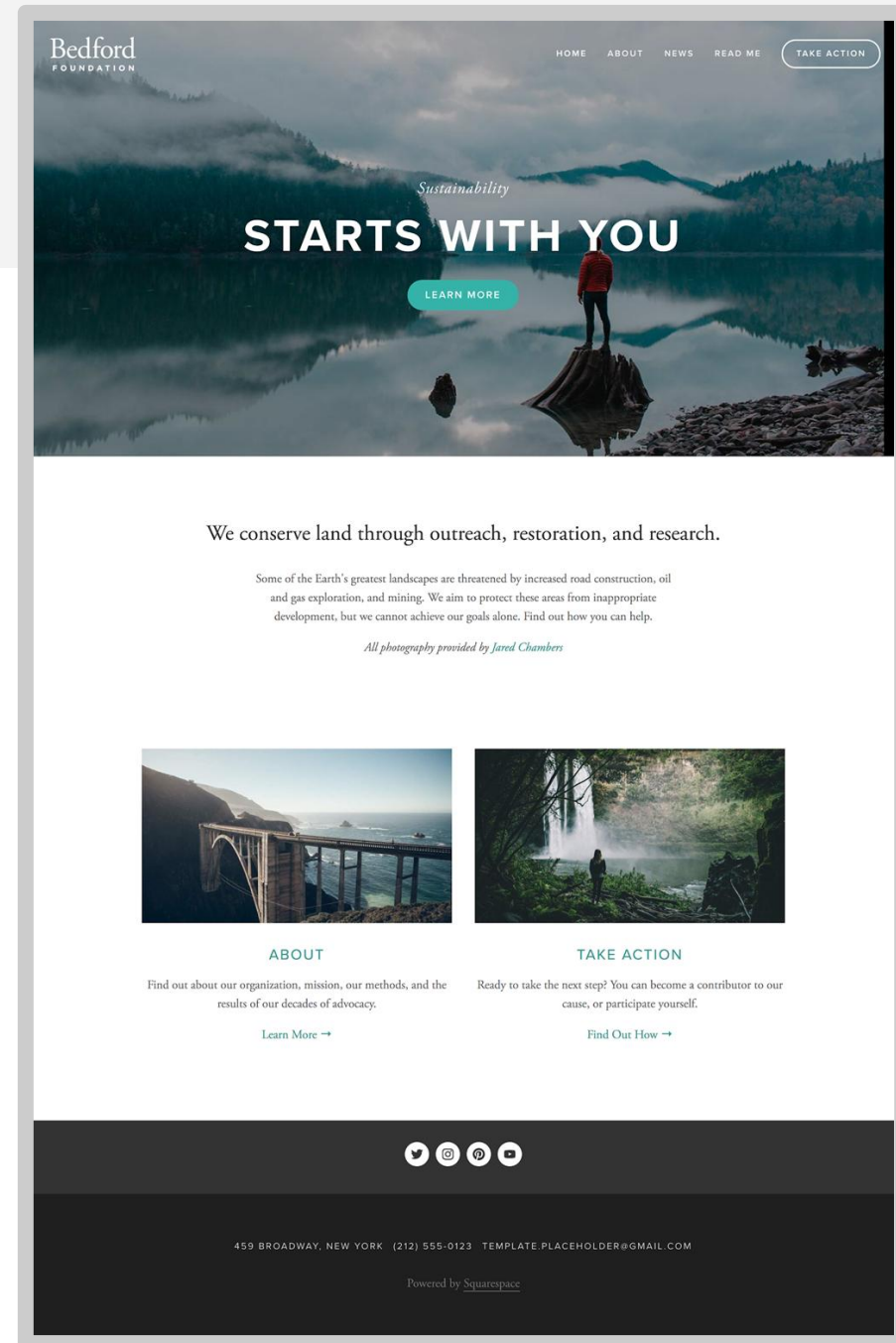
Un template molto popolare su [Squarespace](#) appare [così](#):

Q: siamo in grado di riprodurlo?



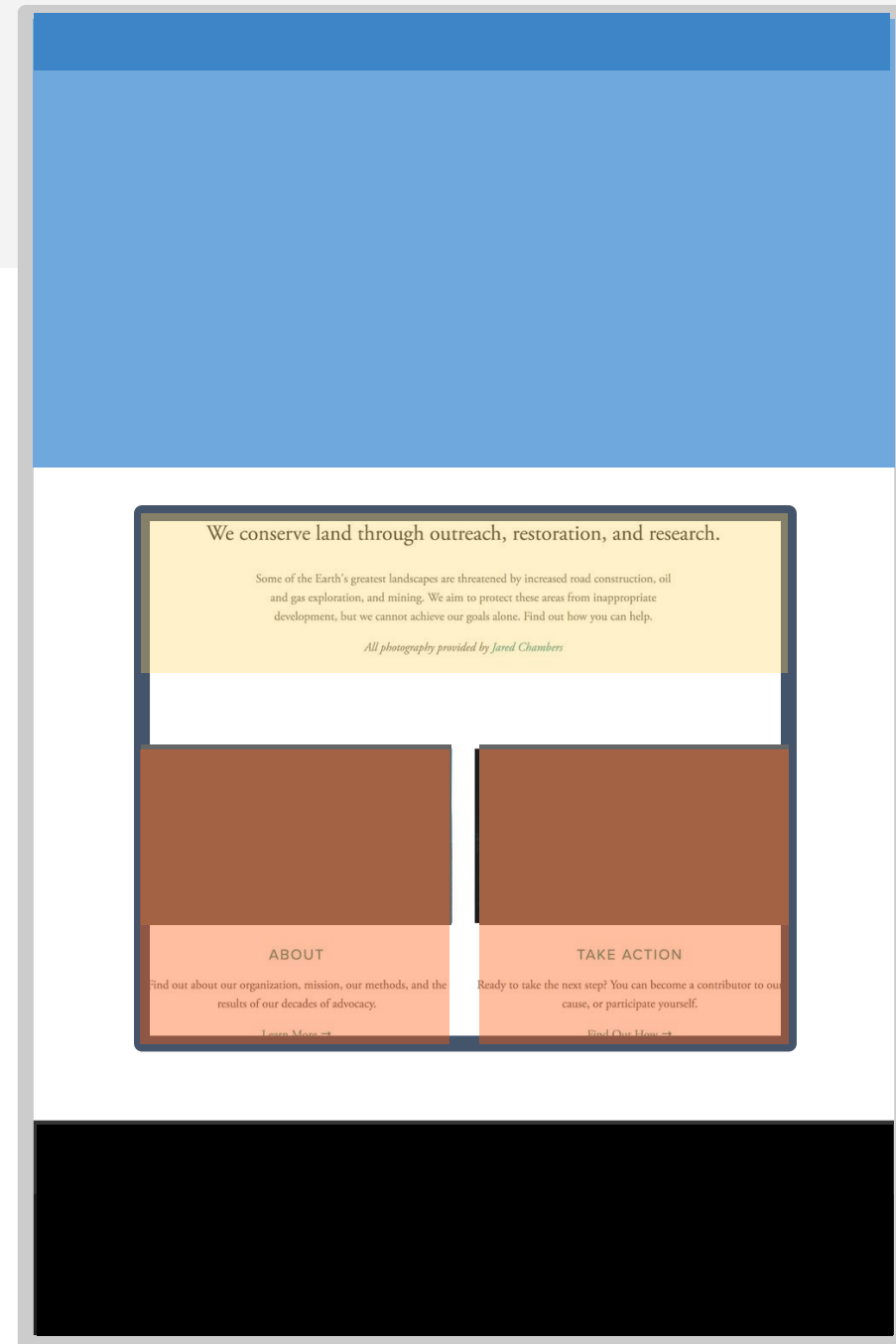
Forma base

Pensiamo in termini di
disposizione dei riquadri



Forma base

Ragioniamo in termini di disposizione dei riquadri:

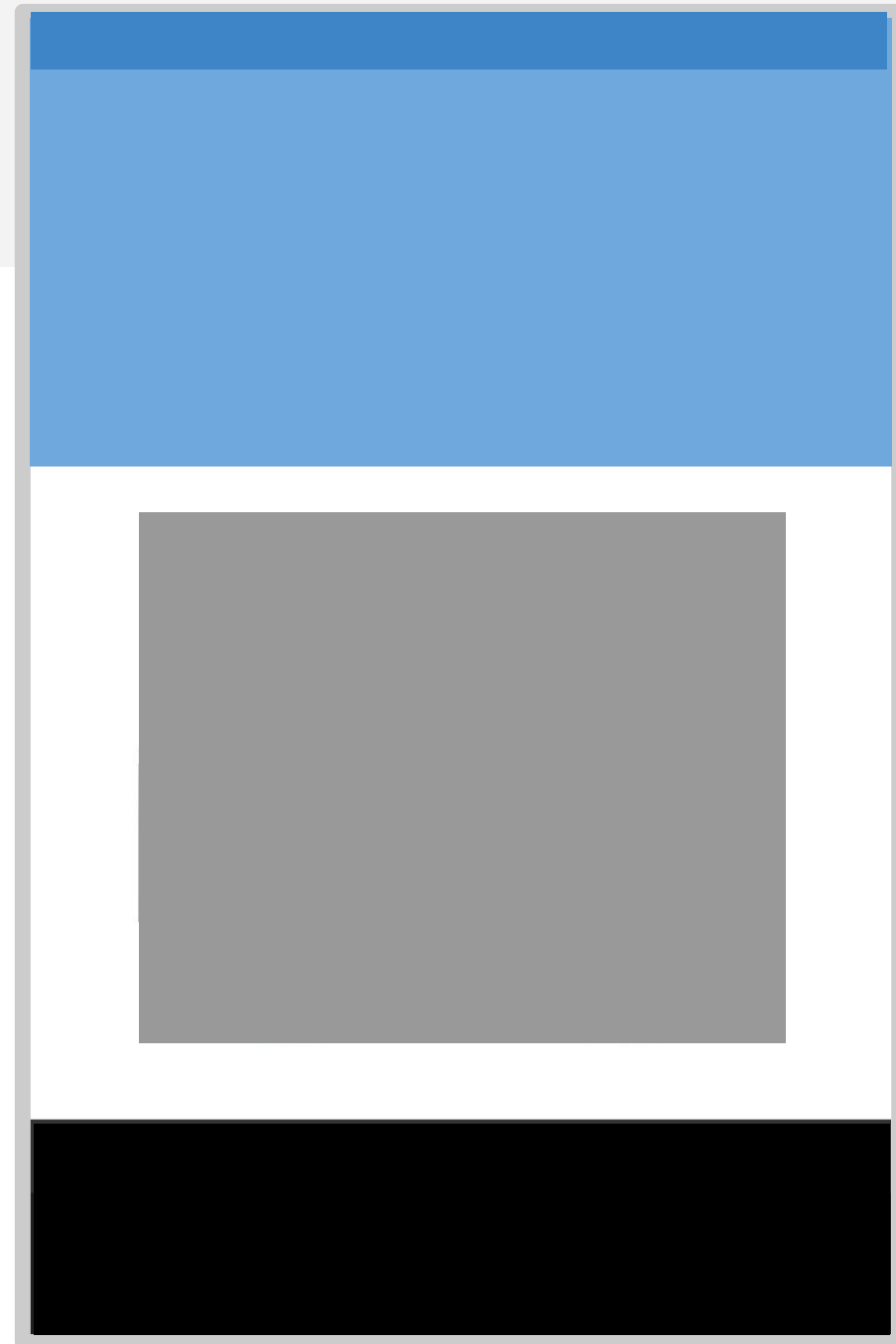


Forma base

Ragioniamo in termini di
disposizione dei riquadri:

Proviamo!

[JsBin](#)



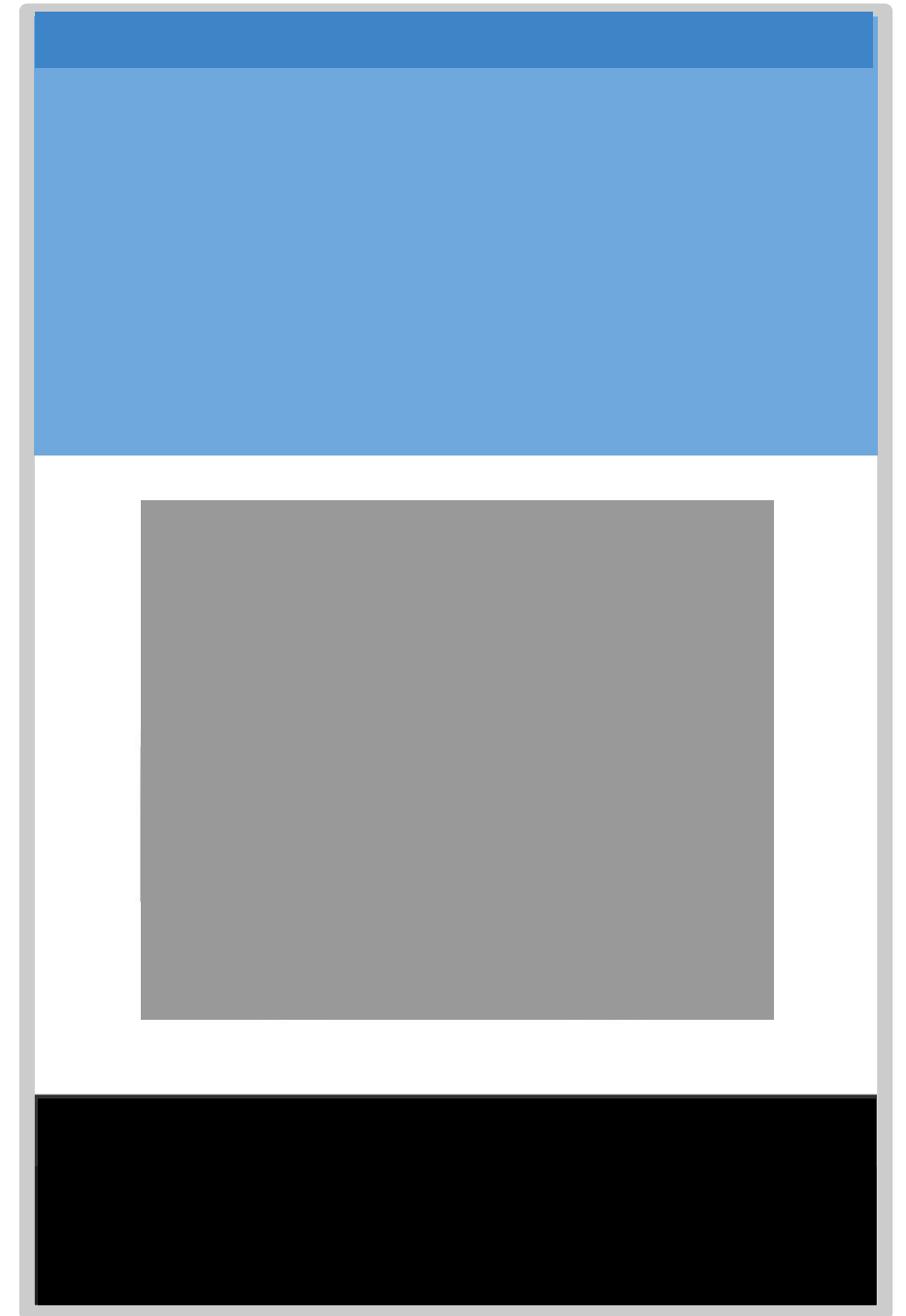
Elementi HTML di sezionamento

Nome	Descrizione
<code><p></code>	Paragrafo (mdn)
<code><h1>-<h6></code>	Intestazioni di sezione (mdn)
<code><article></code>	Una pagina (mdn) Di solito, è il primo elemento dopo body .
<code><section></code>	Sezione generica di un documento (mdn)
<code><header></code>	Sezione introduttiva di un documento (mdn)
<code><footer></code>	Piè di pagina di un documento o di una sezione (mdn)
<code><nav></code>	Sezione di navigazione (mdn)

Questi elementi HTML non "fanno" niente; sono più descrittivi di generici `<div>`. Rendono l'HTML più leggibile. Vedi [MDN](#) per maggiori info.

Iniziamo con una struttura HTML semplificata

```
<html>
<head>
  <meta charset="utf-8">
  <title>Squarespace layout</title>
</head>
<body>
  <nav></nav>
  <header></header>
  <section>
    <!-- main paragraph -->
    <p></p>
    <!-- Two column-type things -->
    <div id="flex-container">
      </div>
  </section>
  <footer></footer>
</body>
</html>
```



Header

Navbar:

- Height: 75px
- Background: royalblue
- `<nav>`

Header:

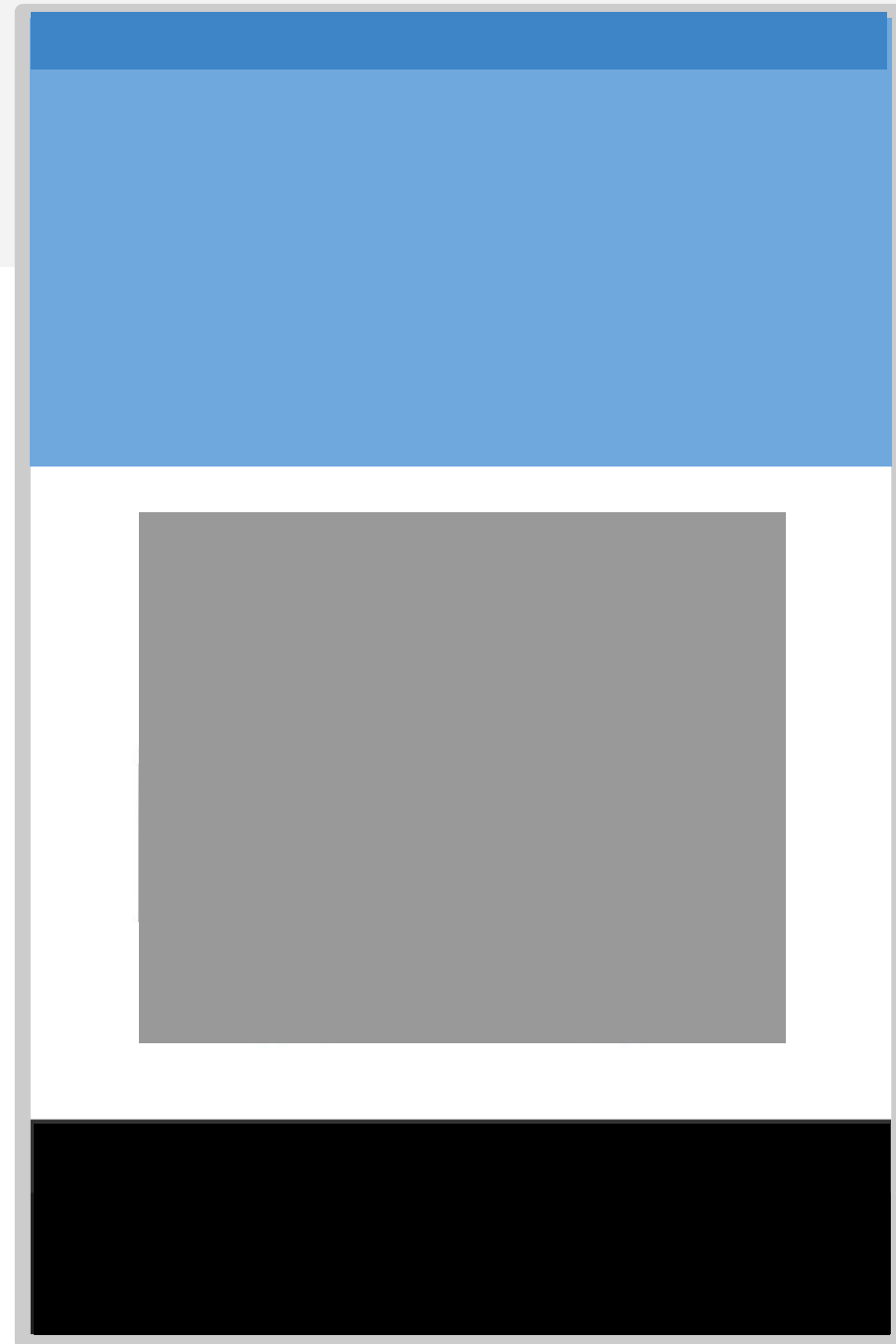
- Height: 400px;
- Background: lightskyblue
- `<header>`



Sezione principale

Riquadro grigio:

- Spazio circostante:
75px sopra e sotto;
100px ds e sn
- Height: 500px
- Background: gray
- `<section>`



Piè di pagina

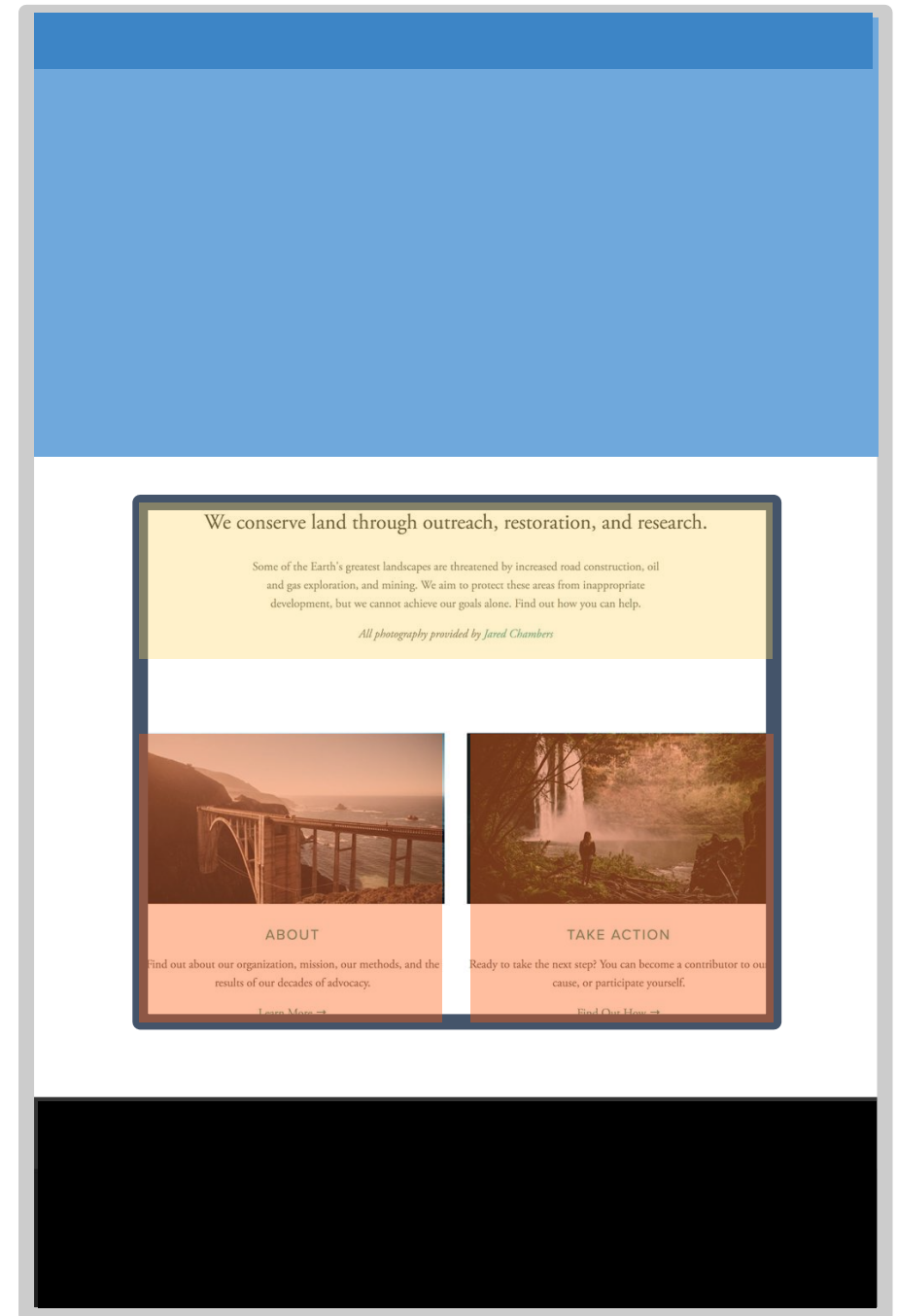
Footer:

- Height: 100px
- Background: Black
- `<footer>`



Struttura HTML completa

```
<html>
<head>
  <meta charset="utf-8">
  <title>Squarespace layout</title>
</head>
<body>
  <nav></nav>
  <header></header>
  <section>
    <!-- main paragraph -->
    <p></p>
    <!-- Two column-type things -->
    <div id="flex-container">
      <div class="col"></div>
      <div class="col"></div>
    </div>
  </section>
  <footer></footer>
</body>
</html>
```



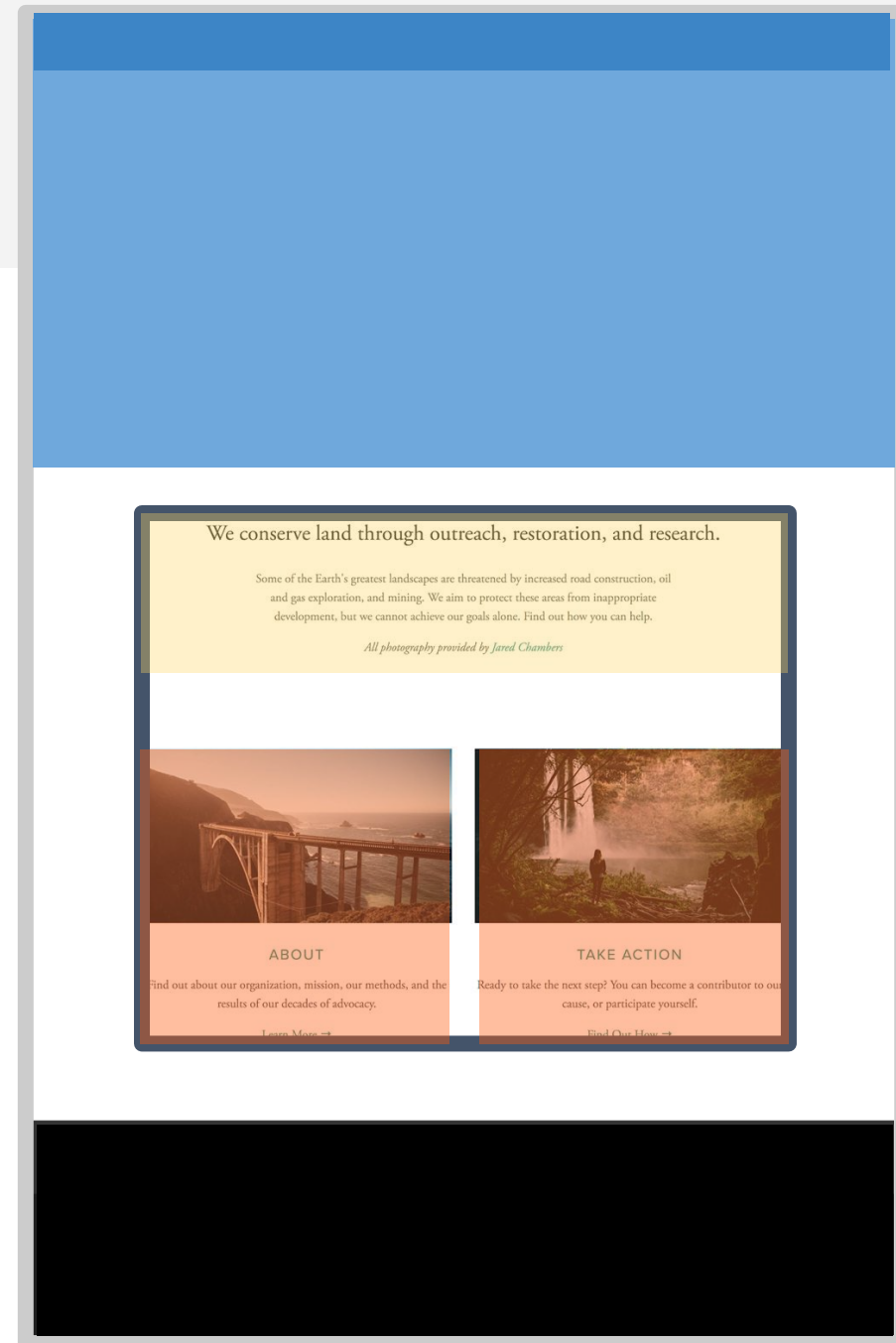
Contenuto principale

Riquadro giallo:

- Height: 200px
- Background: khaki
- Spazio sottostante: 75px
- `<p>`

Riquadro arancione:

- Height: 400px;
- Width: 48% del genitore con spazio in mezzo
- Background: tomato
- `<div>`



Contenuto principale

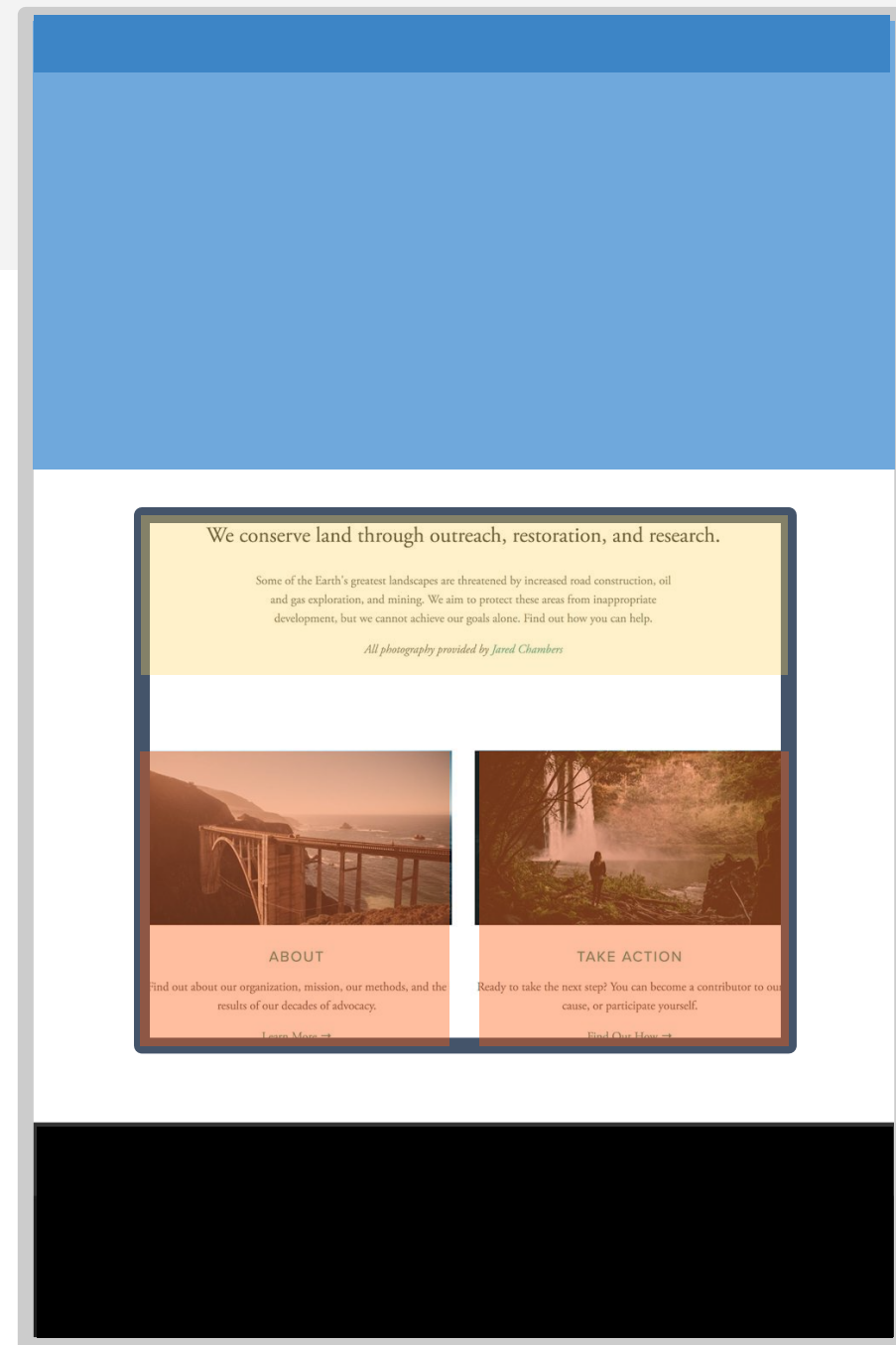
Riquadro giallo:

- Height: 200px
- Background: khaki
- Spazio sottostante: 75px
- `<p>`

Riquadro arancione:

- Height: 400px;
- Width: 48% del genitore con spazio in mezzo
- Background: tomato
- `<div>`

Qui ci piantiamo!



Soluzione HTML + CSS

ma ci sono proprietà CSS che non conosciamo ancora: FLEXBOX

```
<html>
<head>
  <meta charset="utf-8">
  <title>Squarespace layout</title>
</head>
<body>
  <nav></nav>
  <header></header>
  <section>
    <!-- main paragraph -->
    <p></p>
    <!-- Two column-type things -->
    <div id="flex-container">
      <div class="col"></div>
      <div class="col"></div>
    </div>
  </section>
  <footer></footer>
</body>
</html>
```

```
body {
  margin: 0;
}

#flex-container {
  display: flex;
  justify-content:
space-between;
  height: 200px;
}

nav {
  background-color:
royalblue;
  height: 75px;
}

header {
  background-color:
lightskyblue;
  height: 400px;
}

section {
  background-color: gray;
  margin: 75px 100px;
}

footer {
  background-color: black;
  height: 100px;
}

p {
  background-color: khaki;
  height: 200px;
  margin-bottom: 75px;
}

div.col {
  background-color:
tomato;
  border: 2px solid black;
  width: calc(50% - 10px);
}
```

Flexbox: CSS Flexible Box Layout

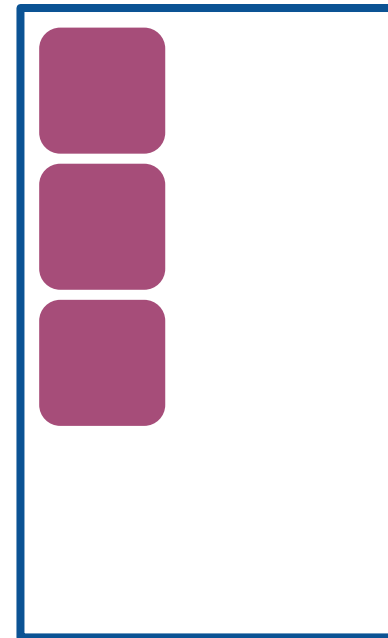
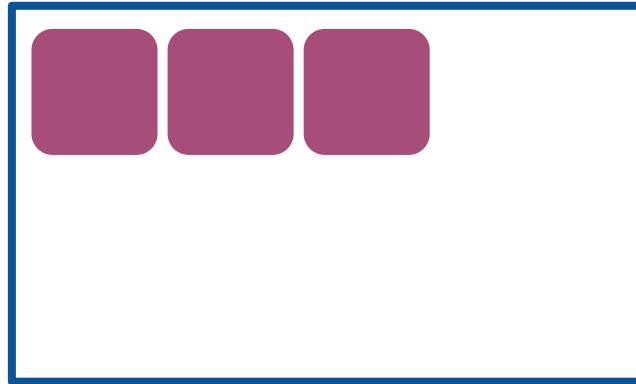
Disposizione dei riquadri (fino ad ora)



Flexbox

Per layout più sofisticati, possiamo usare riquadri flessibili: **flexbox**

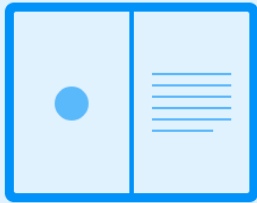
La proprietà **flexbox** permette di disporre in maniera flessibile i riquadri in righe o in colonne



Flexbox

Flexbox permette disposizioni molto articolate.

- Alcuni esempi:



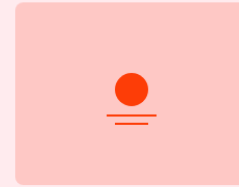
Split-screen



Sidebar



Sticky footer



Centering



Fluid grid



Collection grid

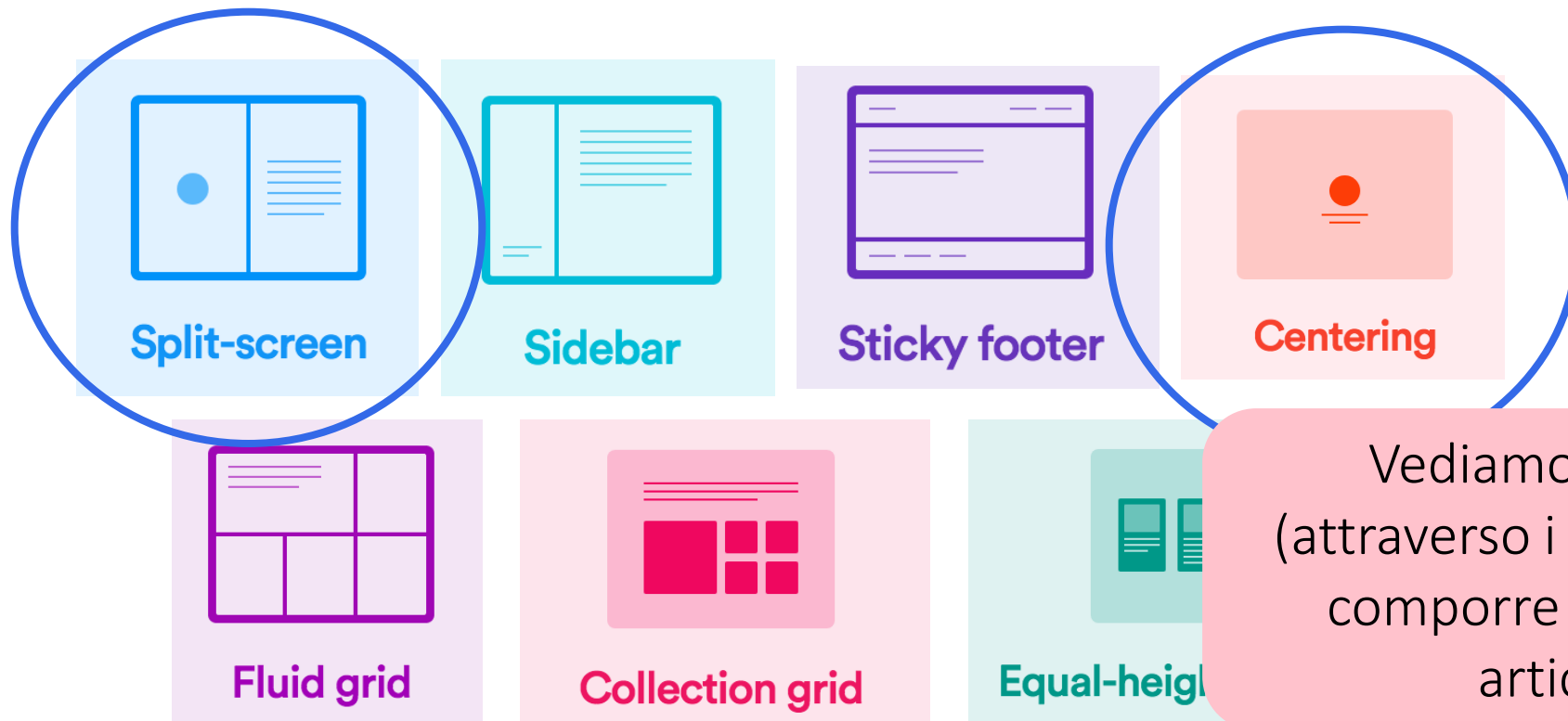


Equal-height modules

Flexbox

Flexbox permette disposizioni molto articolate.

- Alcuni esempi:



Vediamo i casi base
(attraverso i quali possiamo
comporre soluzioni più
articolate)

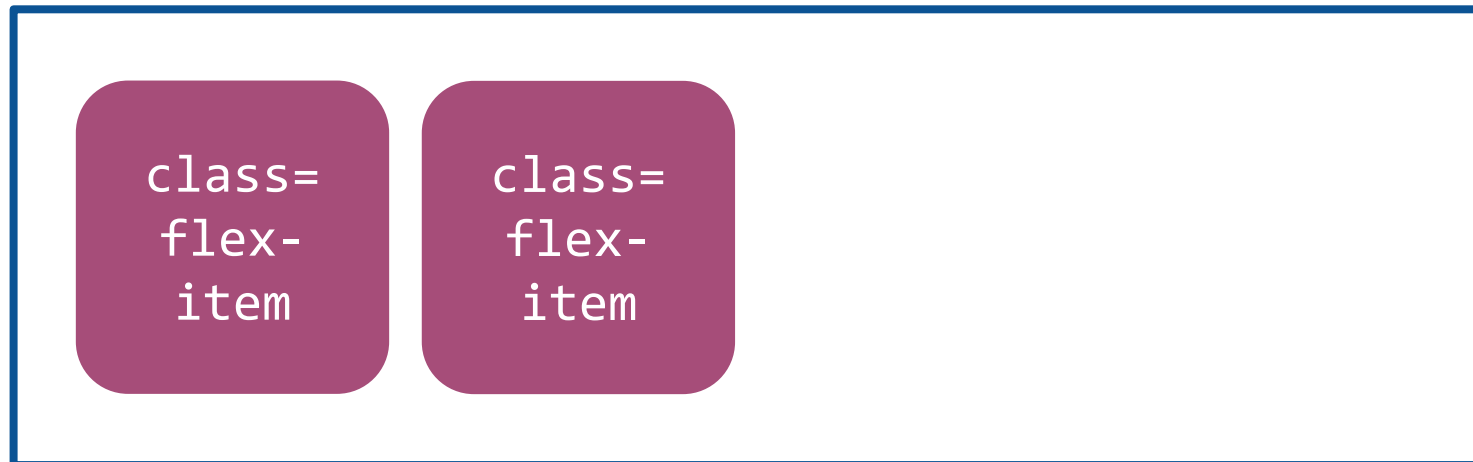
Flexbox basics

Due elementi principali:

- un **Flex container**, che contiene:
- uno o più **Flex item**

Applichiamo proprietà CSS al **flex container** per specificare come devono essere disposti i suoi flex item

id=flex-container



NB: i nomi dell'id e della classe sono arbitrari. Quello che conta sono le proprietà CSS che gli associamo

Flexbox basics

Per definire un flex container, impostiamo la proprietà **display**.

Esistono due tipi di container:

- Block container:
`display: flex;`
il prossimo container scorre sotto



- Inline container:
`display: inline-flex;`
il prossimo container scorre a ds



Flexbox basics: direzione

In un flex container, i flex item sono disposti su una **direzione**: riga (default) o colonna

- Disposizione in riga:

`display: flex;`



[JsBin](#)

- Disposizione in colonna:

`display: flex;`

`flex-direction: column;`

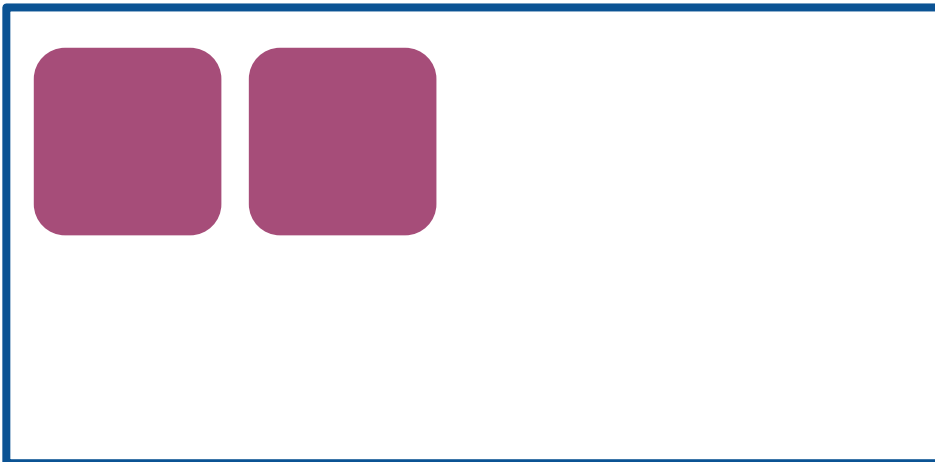


Flexbox basics: **justify-content**

Possiamo specificare come posizionare gli item nel riquadro del container rispetto alla direzione con la proprietà **justify-content**

[JsBin](#)

```
#flex-container {  
  display: flex;  
  justify-content: flex-start;  
}
```

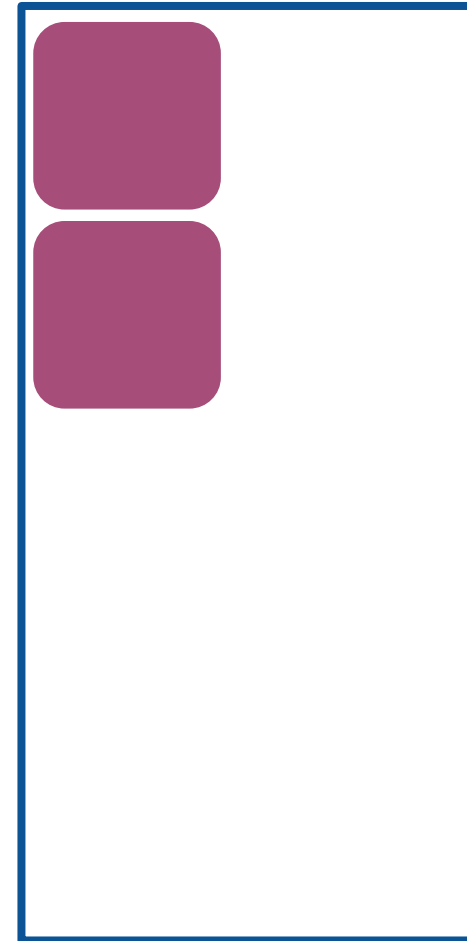


Flexbox basics: **justify-content**

Possiamo specificare come posizionare gli item nel riquadro del container rispetto alla direzione con la proprietà **justify-content**

[JsBin](#)

```
#flex-container {  
  display: flex;  
  justify-content: flex-start;  
  flex-direction: column;  
}
```



Flexbox basics: justify-content

Possiamo specificare come posizionare gli item nel riquadro del container rispetto alla direzione con la proprietà **justify-content**

[JsBin](#)

```
#flex-container {  
  display: flex;  
  justify-content: flex-end;  
}
```

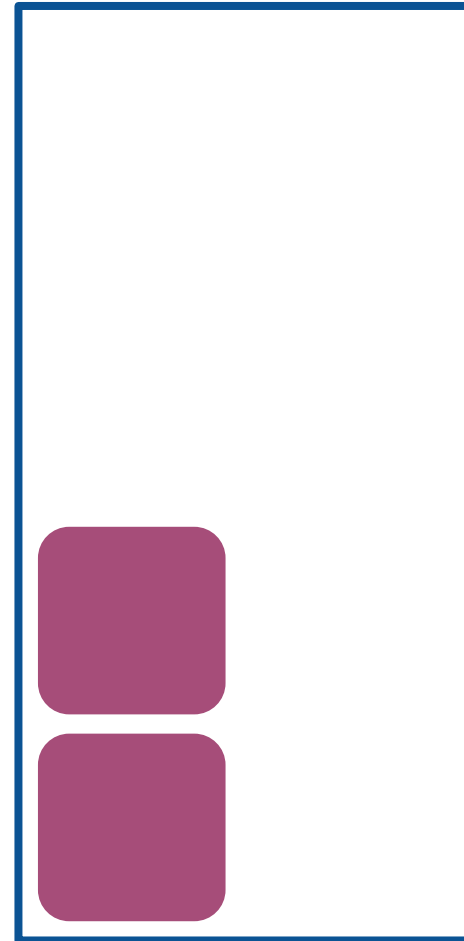


Flexbox basics: **justify-content**

Possiamo specificare come posizionare gli item nel riquadro del container rispetto alla direzione con la proprietà **justify-content**

[JsBin](#)

```
#flex-container {  
  display: flex;  
  justify-content: flex-end;  
  flex-direction: column;  
}
```

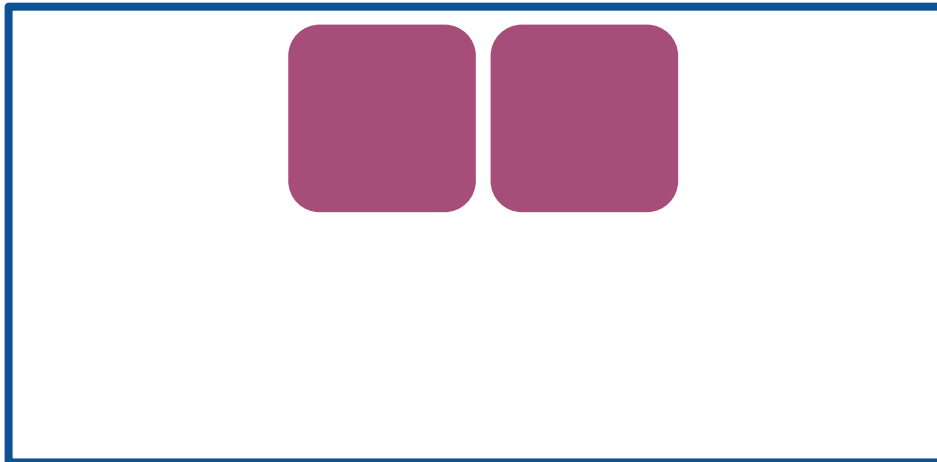


Flexbox basics: **justify-content**

Possiamo impostare come posizionare l'item rispetto alla direzione orizzontale* nel riquadro con **justify-content** nel flex container:

[JsBin](#)

```
#flex-container {  
  display: flex;  
  justify-content: center;  
}
```

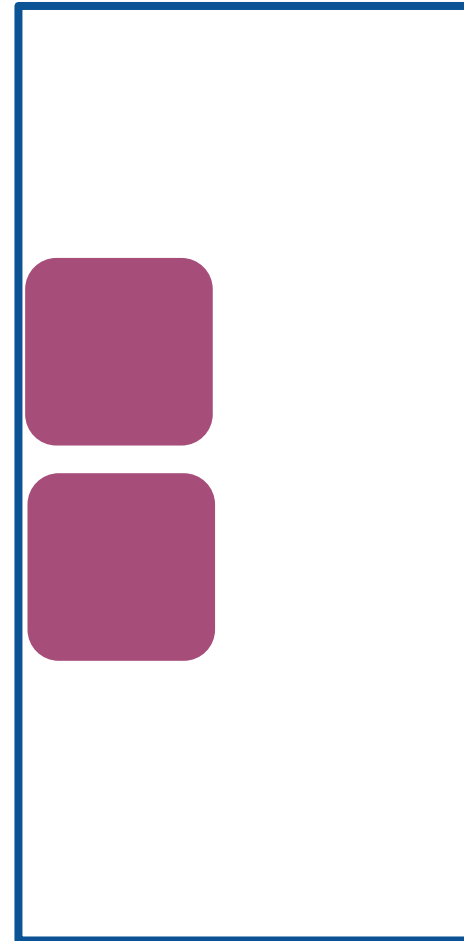


Flexbox basics: **justify-content**

Possiamo specificare come posizionare gli item nel riquadro del container rispetto alla direzione con la proprietà **justify-content**

[JsBin](#)

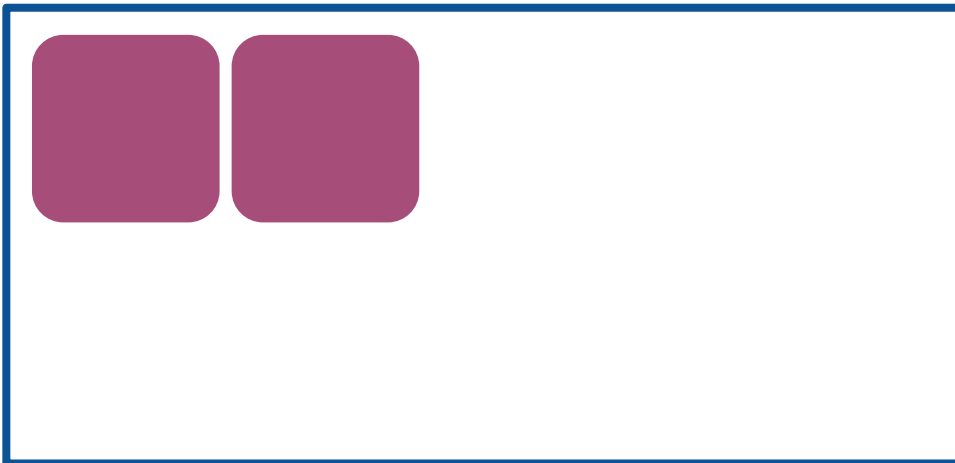
```
#flex-container {  
  display: flex;  
  justify-content: flex-center;  
  flex-direction: column;  
}
```



Flexbox basics: align-items

Possiamo specificare come posizionare gli item nel riquadro del container **rispetto alla perpendicolare della direzione** con la proprietà **align-items**:

```
#flex-container {  
  display: flex;  
  align-items: flex-start;  
}
```

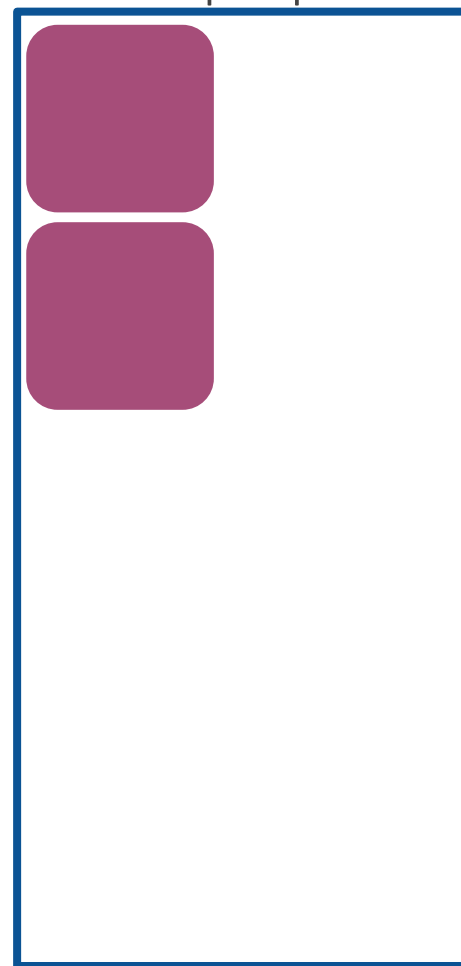
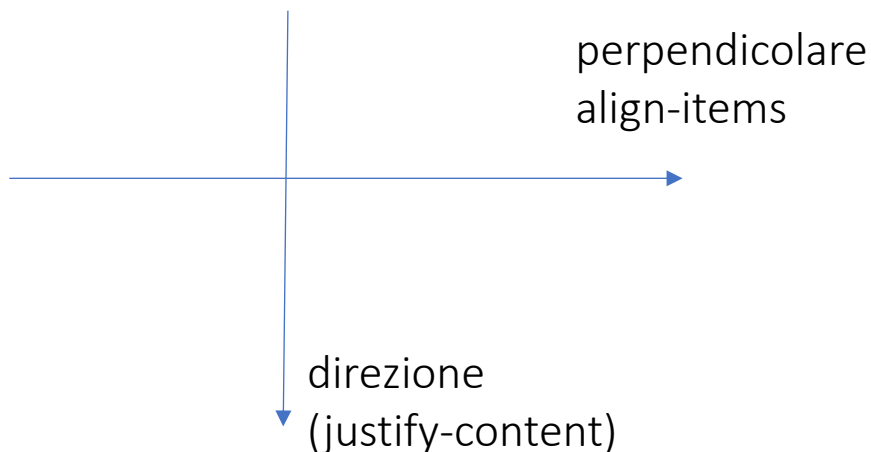


Flexbox basics: align-items

Possiamo specificare come posizionare gli item nel riquadro del container **rispetto alla perpendicolare della direzione** con la proprietà **align-items**:

```
#flex-container {  
  display: flex;  
  align-items: flex-start;  
  flex-direction: column;  
}
```

[JsBin](#)

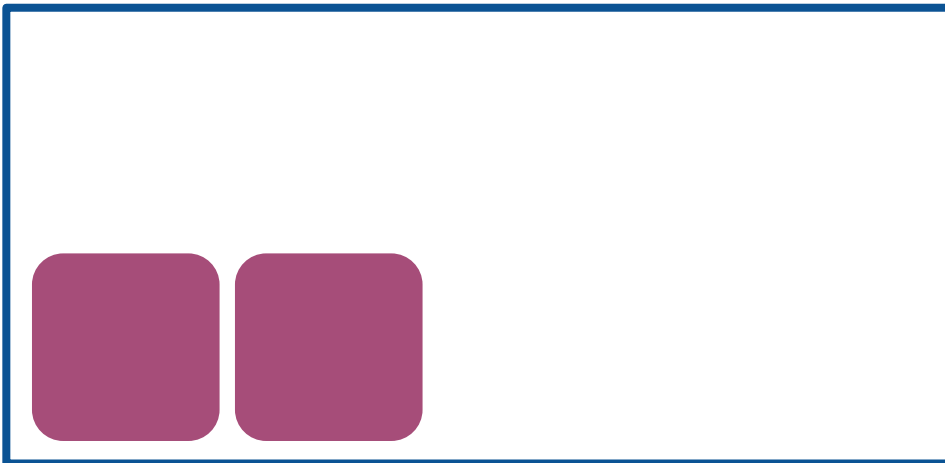


Flexbox basics: **align-items**

Possiamo controllare come posizionare verticalmente l'item nel riquadro con **align-items** nel flex container:

[JsBin](#)

```
#flex-container {  
  display: flex;  
  align-items: flex-end;  
}
```

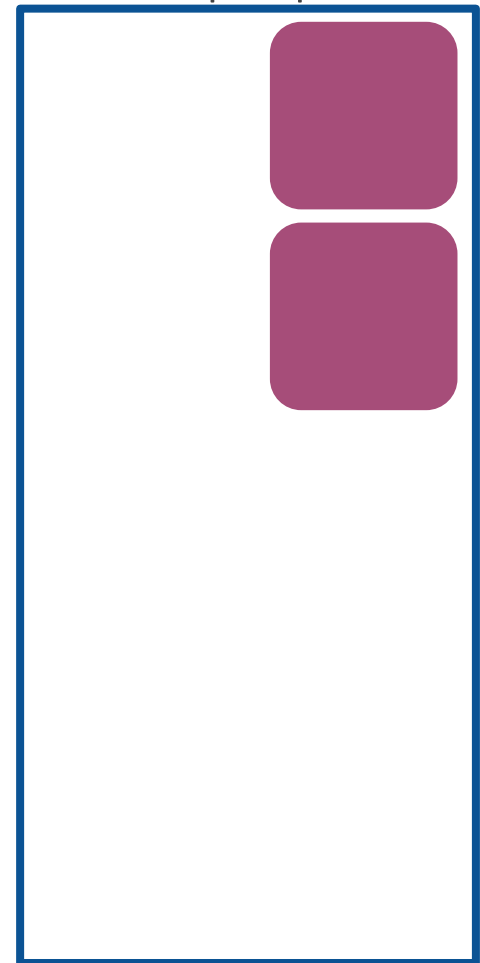
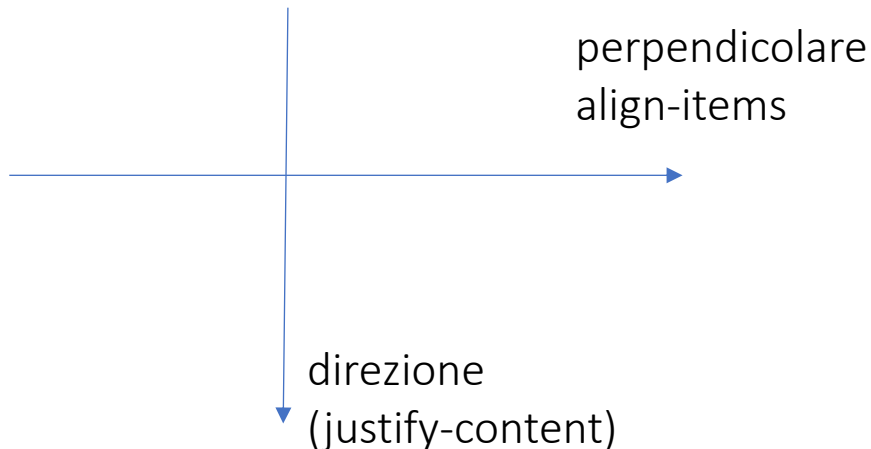


Flexbox basics: align-items

Possiamo specificare come posizionare gli item nel riquadro del container **rispetto alla perpendicolare della direzione** con la proprietà **align-items**:

```
#flex-container {  
  display: flex;  
  align-items: flex-end;  
  flex-direction: column;  
}
```

[JsBin](#)

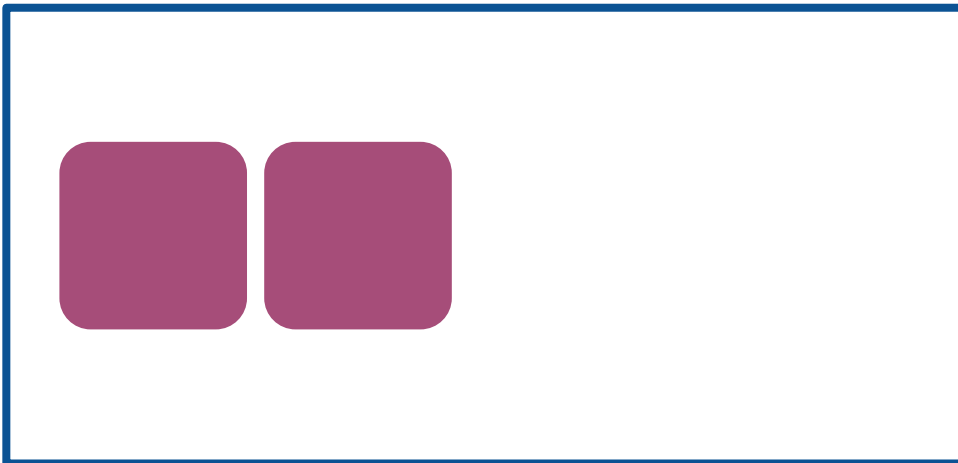


Flexbox basics: align-items

Possiamo controllare come posizionare verticalmente (rispetto alla direzione) gli item nel riquadro con **align-items** nel flex container:

```
#flex-container {  
  display: flex;  
  align-items: center;  
}
```

[JsBin](#)

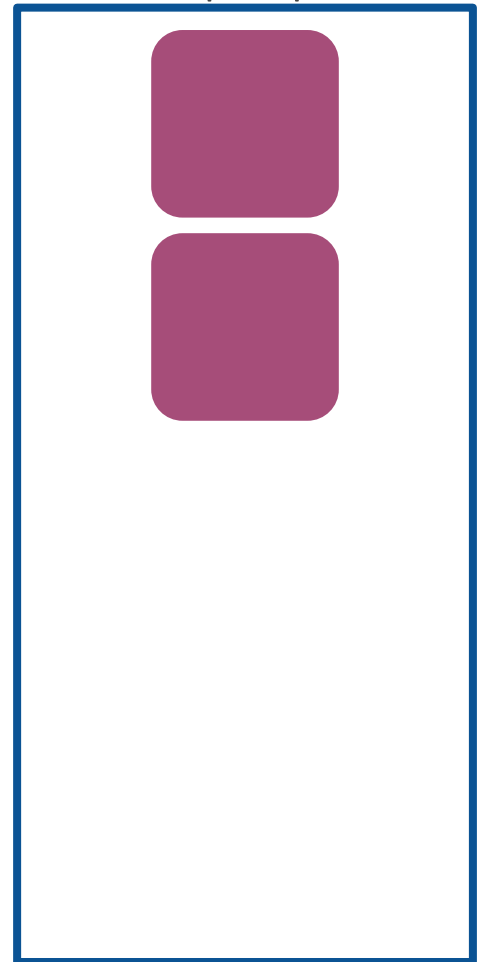
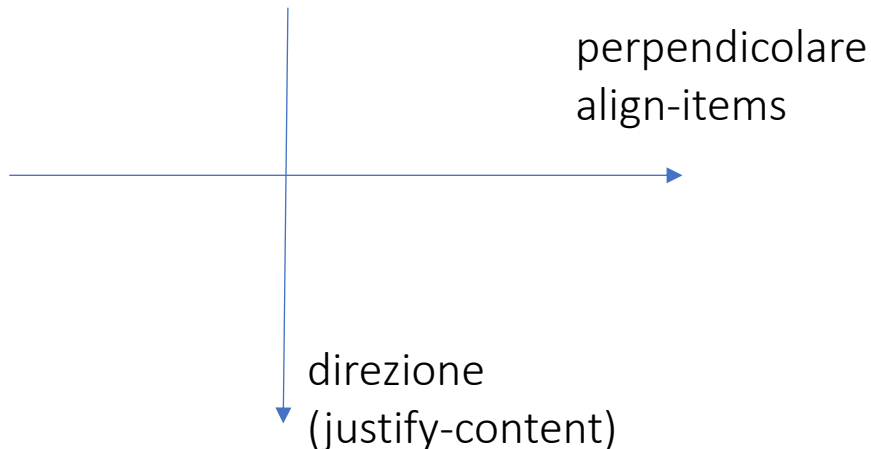


Flexbox basics: align-items

Possiamo specificare come posizionare gli item nel riquadro del container **rispetto alla perpendicolare della direzione** con la proprietà **align-items**:

```
#flex-container {  
  display: flex;  
  align-items: center;  
  flex-direction: column;  
}
```

[JsBin](#)



Item multipli

Stesse regole se abbiamo più item:

```
#flex-container {  
  display: flex;  
  justify-content: flex-start;  
  align-items: center;  
}
```

[JsBin](#)



Item multipli

Stesse regole se abbiamo più item:

```
#flex-container {  
  display: flex;  
  justify-content: flex-end;  
  align-items: center;  
}
```

[JsBin](#)



Item multipli

Stesse regole se abbiamo più item:

```
#flex-container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

[JsBin](#)

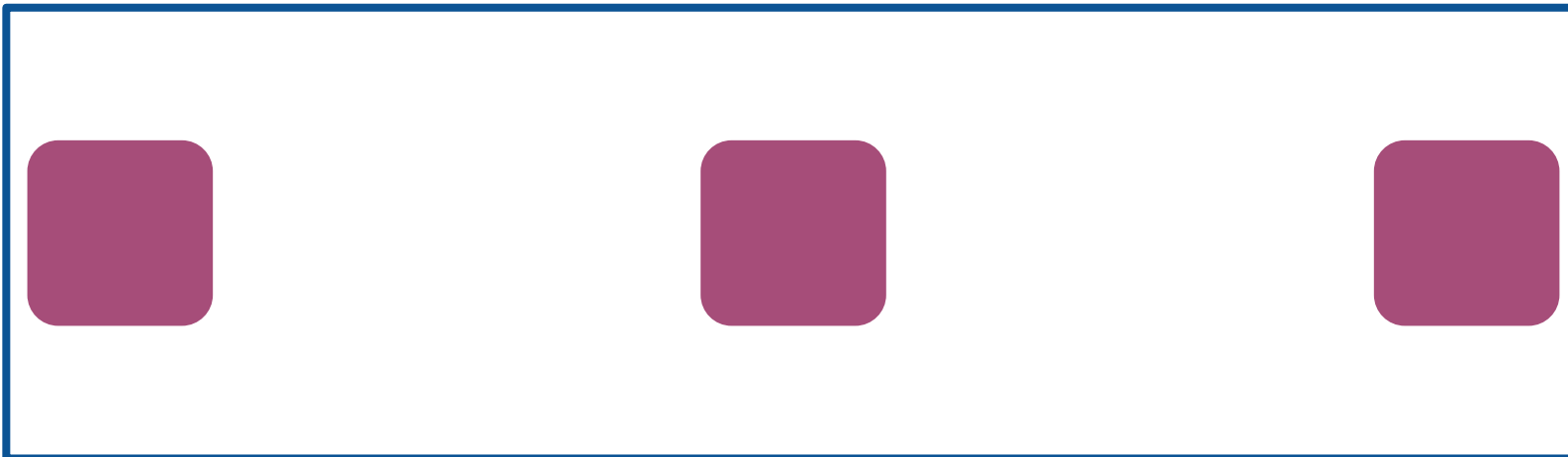


Spaziatura di item multipli

space-between e **space-around** permettono di gestire lo spazio **tra** e **intorno** agli item all'interno del contenitore:

```
#flex-container {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

[JsBin](#)

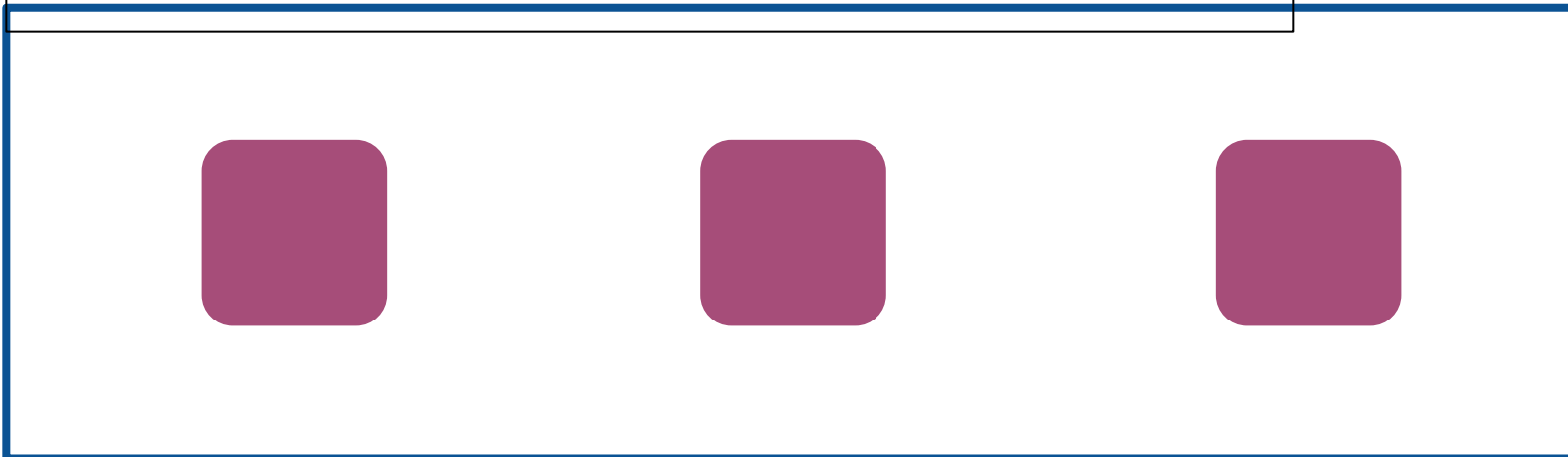


Item multipli

space-between e **space-around** permettono di gestire lo spazio **tra** e **intorno** agli item all'interno del contenitore:

```
#flex-container {  
  display: flex;  
  justify-content: space-around;  
  align-items: center;  
}
```

[JsBin](#)



Prima di continuare...

Cosa succede se il flex item è un elemento inline?

HTML

```
<html>
  <head>
    <meta charset="utf-8">
    <title>Flexbox example</title>
  </head>
  <body>

    <div id="flex-container">
      <span class="flex-item"></span>
      <span class="flex-item"></span>
      <span class="flex-item"></span>
    </div>

  </body>
```

CSS

```
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  height: 50px;
  width: 50px;
  margin: 5px;
}
```

???

[JsBin](#)

HTML

```
<html>
  <head>
    <meta charset="utf-8">
    <title>Flexbox example</title>
  </head>
  <body>

    <div id="flex-container">
      <span class="flex-item"></span>
      <span class="flex-item"></span>
      <span class="flex-item"></span>
    </div>

  </body>
```

CSS

```
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  height: 50px;
  width: 50px;
  margin: 5px;
}
```



Ricordiamo block vs inline

Se #flex-container **non è impostato** display: flex:

[JsBin](#)



```
HTML
<html>
  <head>
    <meta charset="utf-8">
    <title>Flexbox example</title>
  </head>
  <body>
    <div id="flex-container">
      <span class="flex-item"></span>
      <span class="flex-item"></span>
      <span class="flex-item"></span>
    </div>
  </body>
</html>

CSS
#flex-container {
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  height: 50px;
  width: 50px;
  margin: 5px;
}
```

Gli elementi **span.flex-items** non sarebbero visualizzati: siccome sono vuoti, essendo inline, non avrebbero altezza e larghezza

Flexbox layout

[JsBin](#)

```
HTML
<html>
  <head>
    <meta charset="utf-8">
    <title>Flexbox example</title>
  </head>
  <body>

    <div id="flex-container">
      <span class="flex-item"></span>
      <span class="flex-item"></span>
      <span class="flex-item"></span>
    </div>

  </body>
</html>

CSS
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  height: 50px;
  width: 50px;
  margin: 5px;
}
```



Perchè cambia se **display: flex**?

Sono ancora elemento inline, eppure hanno altezza e larghezza

Flexbox layout

- Quando impostiamo un elemento container `display: flex`, i figli diretti sono **flex items** e seguono un nuovo insieme di regole.
- **Flex items non sono né block né inline**; hanno un insieme di regole diverse per altezza, larghezza, disposizione.
 - I contenuti di un flex item seguono le regole block/inline, relativamente al bordo.
- Le dimensioni (**height** e **width**) dei flex items seguono regole articolate

Vediamo su [JsBin](#)

Dimensionare i flex item

Dimensioni flex item

I flex items hanno una larghezza* iniziale, che per default è:

- La larghezza del contenuto
- Il valore impostato esplicitamente con **width**
- Il valore impostato con **flex-basis**

Il valore iniziale della larghezza* di un flex item viene chiamato **flex basis**.

*larghezza nel caso di righe;
altezza nel caso di colonne

Dimensioni flex item: larghezza iniziale

I flex items hanno una larghezza* iniziale, definita **flex basis**, che vale:

- Larghezza del contenuto
- Il valore impostato esplicitamente con **width**

La larghezza* esplicita di un flex item è rispettata *per tutti i flex items*, siano questi inline, block, o inline-block.

*larghezza nel caso di righe;
altezza nel caso di colonne

Dimensioni flex item

Se togliamo **height** e **width**, I nostri flex items spariscono, perchè **flex basis** ora è la dimensione del contenuto, che è vuoto:

[JsBin](#)

```
HTML
<title>Flexbox example</title>
</head>
<body>

  <div id="flex-container">
    <span class="flex-item"></span>
    <div class="flex-item"></div>
    <span class="flex-item"></span>
  </div>

</body>
</html>

CSS
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  margin: 5px;
}
```

Vediamo su [JsBIN](#)



flex-shrink

La larghezza* di un flex item può compattarsi (riducendola rispetto a **flex basis**) per adattare la disposizione degli item alle dimensioni del container attraverso la proprietà **flex-shrink** :

flex-shrink:

- 1, il flex item si compatta per stare dentro il container.
- 0, mantiene le sue dimensioni.

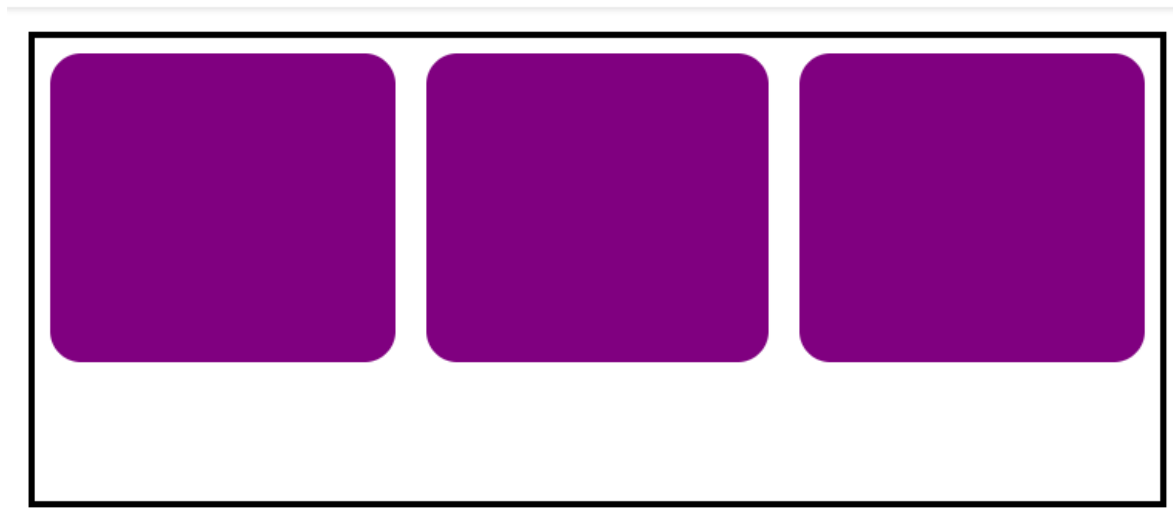
flex-shrink: 1 per default.

*larghezza nel caso di righe;
altezza nel caso di colonne

```
#flex-container {  
  display: flex;  
  align-items: flex-start;  
  border: 2px solid black;  
  height: 150px;  
}
```

```
.flex-item {  
  width: 500px;  
  height: 100px;  
  
  border-radius: 10px;  
  background-color: purple;  
  margin: 5px;  
}
```

[JsBin](#)



Gli elementi si comprimono per aderire alle dimensioni del container.

[JsBin](#)

```
#flex-container {  
  display: flex;  
  align-items: flex-start;  
  border: 2px solid black;  
  height: 150px;  
}
```

```
.flex-item {  
  width: 500px;  
  height: 100px;  
  flex-shrink: 0;  
  
  border-radius: 10px;  
  background-color: purple;  
  margin: 5px;  
}
```

Impostando `flex-shrink: 0;`



flex-grow

E' il complementare di shrink: attraverso la proprietà **flex-grow**, la larghezza* del flex item può adattarsi alle dimensioni del container **crescendo**:

flex-grow:

- 1, il flex item cresce per occupare tutto lo spazio possibile del contenitore
- 0, non cresce

flex-grow: 0 per default.

[JsBin](#)

*width in the case of rows; height in the case of columns

flex-grow esempio

Togliamo height e width dai nostri flex item:

[JsBin](#)

HTML

```
<title>Flexbox example</title>
</head>
<body>

  <div id="flex-container">
    <span class="flex-item"></span>
    <div class="flex-item"></div>
    <span class="flex-item"></span>
  </div>

</body>
</html>
```

CSS

```
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  margin: 5px;
}
```

flex-grow esempio

Se impostiamo **flex-grow: 1**, i flex items riempiono lo spazio:

[JsBin](#)

```
HTML
<title>Flexbox example</title>
</head>
<body>

  <div id="flex-container">
    <span class="flex-item"></span>
    <div class="flex-item"></div>
    <span class="flex-item"></span>
  </div>

</body>
</html>

CSS
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}

.flex-item {
  border-radius: 10px;
  flex-grow: 1;
  background-color: purple;
  margin: 5px;
}
```



flex-grow distribuzione non uniforme

Impostando valori diversi di **flex-grow** è possibile ottenere distribuzioni diverse della larghezza degli elementi

[JsBin](#)

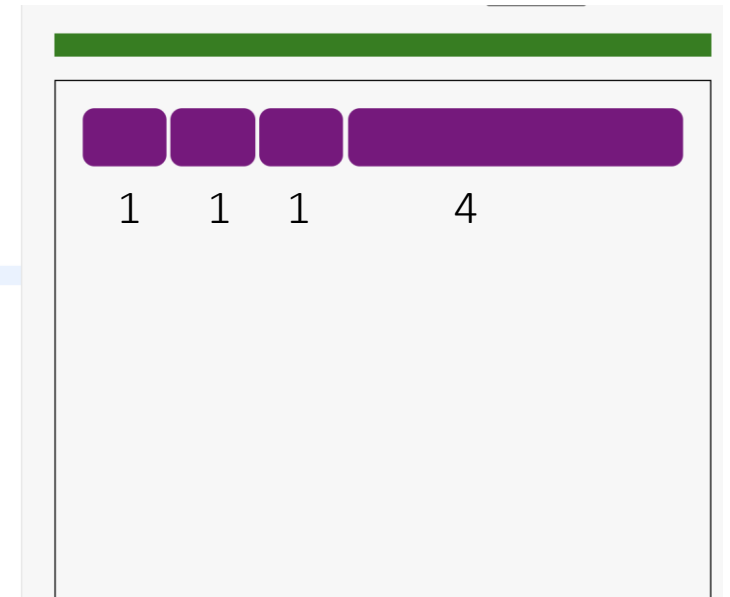
```
<html>
<head>
  <meta name="description" content="flexbox example">
  <meta charset="utf-8">
  <title>Flexbox example</title>
</head>
<body>
  <div class="section"></div>
  <div id="flex-container">
    <div class="flex-item"></div>
    <div class="flex-item special"></div>
    <div class="flex-item"></div>
    <span class="flex-item"></span>
  </div>
</body>
</html>
```

```
.section {
  height: 20px;
  background-color: green;
  margin: 20px;
}

#flex-container {
  margin: 20px;
  display: flex;
  border: 2px solid black;
  padding: 20px;
  height: 600px;
  align-items: stretch;
  #flex-direction: column;
  justify-content: space-between;
}

.flex-item {
  border-radius: 10px;
  background-color: purple;
  margin: 2px;
  #width: 50px;
  height: 50px;
  flex-grow: 1;
}

span.flex-item {
  flex-grow: 4;
}
```



Flex item height**?!

flex-grow controlla solo la larghezza* degli item.

E' possibile agire anche sulla altezza**?

```
HTML
<title>Flexbox example</title>
</head>
<body>

  <div id="flex-container">
    <span class="flex-item"></span>
    <div class="flex-item"></div>
    <span class="flex-item"></span>
  </div>

</body>
</html>

CSS
#flex-container {
  display: flex;
  border: 2px solid black;
  height: 150px;
}
.flex-item {
  border-radius: 10px;
  flex-grow: 1;
  background-color: purple;
  margin: 5px;
}
```



*larghezza nel caso di righe; altezza nel caso di colonne

** altezza nel caso di righe; larghezza nel caso di colonne

align-items: stretch;

Il valore di default value di **align-items** è **stretch**, che significa che ogni item cresce verticalmente* per riempire il contenitore.

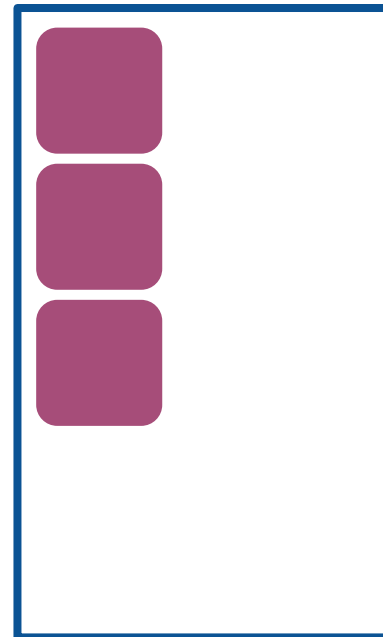
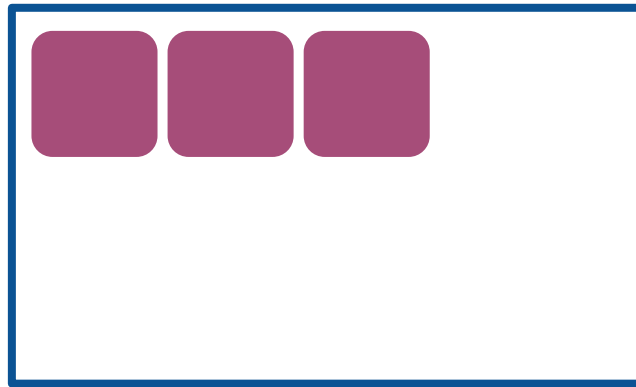
(Nel caso in cui la proprietà **height** dell'item non sia impostata)



*verticalmente nel caso di righe;
orizzontalmente nel caso di colonne

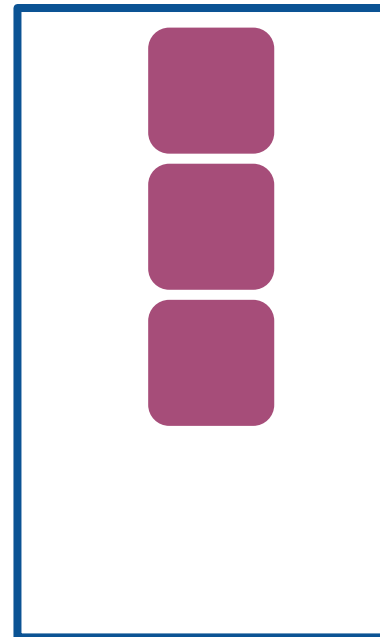
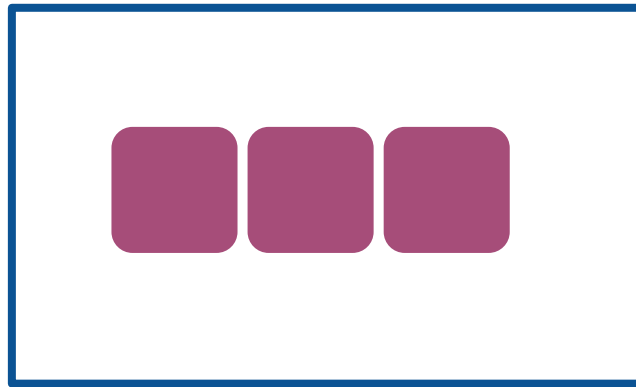
Flex layout recap

- Con **display: flex**, l'elemento diventa un **flex container** e i suoi figli **flex items**.
- Gli item di un flex si dispongono in una riga o in una colonna, impostando la **proprietà flex-direction** del container.



Flex layout recap

- **justify-content** distribuisce gli items orizzontalmente (se `flex-direction: row`), verticalmente (se `flex-direction: column`)
- **align-items** distribuisce gli items verticalmente (se `flex-direction: row`), orizzontalmente (se `flex-direction: column`)



Flex layout recap

Con **flex-direction: row**:

- **flex basis** è la larghezza iniziale di un flex item
 - Esplicitamente imposta con **width**, **flex-basis**, o il contenuto dell'elemento
- La larghezza di un item si riduce per far rientrare tutti gli item nel contenitore se **flex-shrink** è impostato a 1
- La larghezza di un item cresce per riempire il contenitore se **flex-grow** è impostato a 1



Flex layout recap

Con `flex-direction: row`:

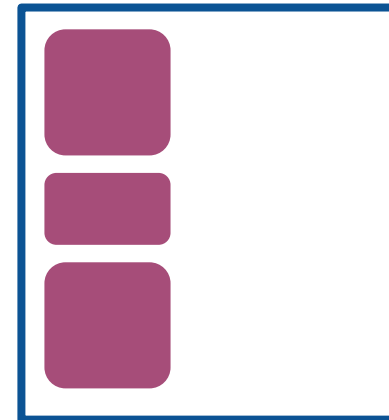
- L'altezza di un flex item è:
 - Quella impostata esplicitamente con `height` sull'item, oppure
 - L'altezza del contenuto dell'item, oppure
 - L'altezza del contenitore se nel contenitore è impostata la proprietà `align-items: stretch`;



Flex layout recap

Per **flex-direction: column**:

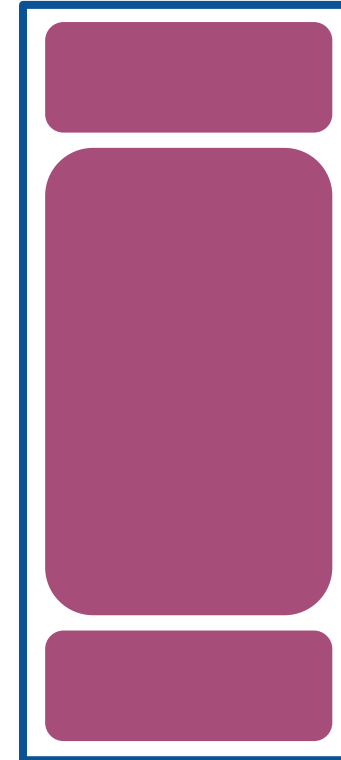
- **flex basis** è l'altezza iniziale di un flex item
 - Quella impostata esplicitamente con **height**, **flex-basis**, or l'altezza del contenuto
- L'altezza di una flex item si riduce per riempire il contenitore se **flex-shrink** è impostato a 1
- L'altezza di una flex item cresce per riempire lo spazio del contenitore se **flex-grow** è impostato a 1



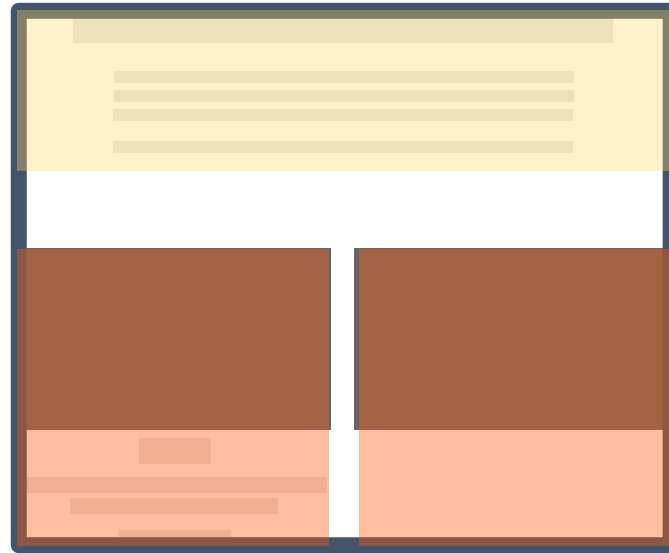
Flex layout recap

Con `flex-direction: column`:

- La larghezza di un flex item è:
 - Quella impostata con `width` nell'item, oppure
 - La larghezza del contenuto dell'item, oppure
 - La larghezza del contenitore se `align-items: stretch`;



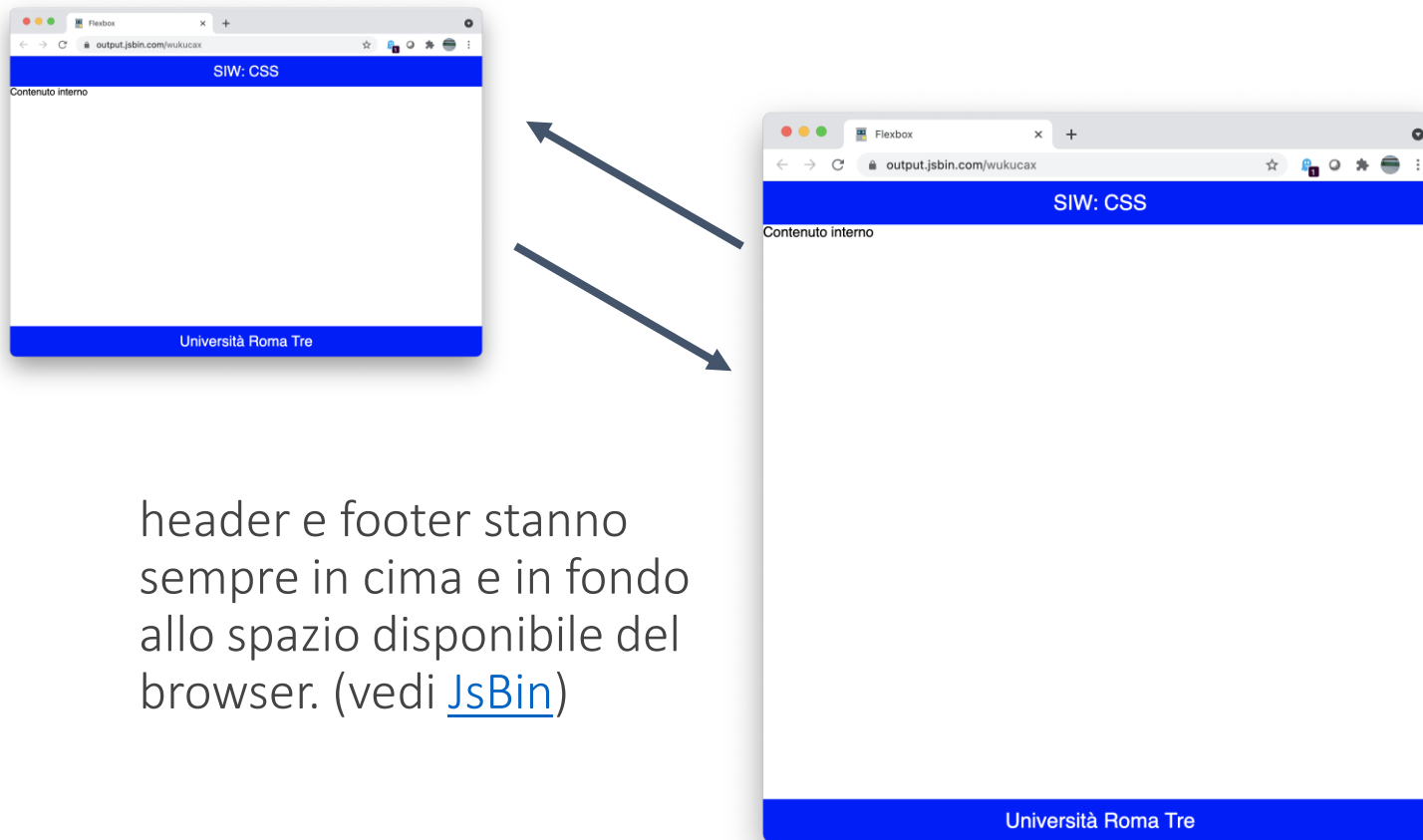
Ora possiamo concludere il nostro esercizio!



Follow along on [Codepen](#)

Problema

Come facciamo un layout come questo? (vedi [JsBin](#))



header e footer stanno
sempre in cima e in fondo
allo spazio disponibile del
browser. (vedi [JsBin](#))

Flexbox: risorse utili

- <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- [https://developer.mozilla.org/en-US/docs/Web/CSS/CSS Flexible Box Layout](https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout)

Valori di `height` e `width` in percentuali

Quando la larghezza (`width`) è definita come una percentuale ([vedi MDS](#)):

- è una percentuale della larghezza dell'elemento genitore

Analogo per l'altezza

Altre proprietà CSS per la
gestione del layout:
position

position

Permette di definire dove posizionare un elemento ([mdn](#)).

Impostazioni:

1. definizione di un metodo di posizionamento, con la proprietà **position**:
static, fixed, absolute, relative
2. definizione dello scostamento (offset) dell'elemento rispetto al metodo di posizionamento con le proprietà: **top, left, bottom, and right**
3. (opzionale) definizione della proprietà **z-index** per strati sovrapposti ([mdn](#))

position

Proprietà: **top, left, bottom, right**

Definiscono lo scostamento (offset) dell'elemento.

Il significato di questi valori dipende dal riferimento, che viene definito attraverso la proprietà **position**:

- **static**: nessun punto di riferimento (*è il default*)
- **fixed**: il riferimento è l'area di visualizzazione della pagina (viewport) del browser
- **absolute**: il riferimento è l'elemento genitore
- **relative**: il riferimento è la posizione statica dell'elemento (dove si troverebbe naturalmente senza offset)

position: static

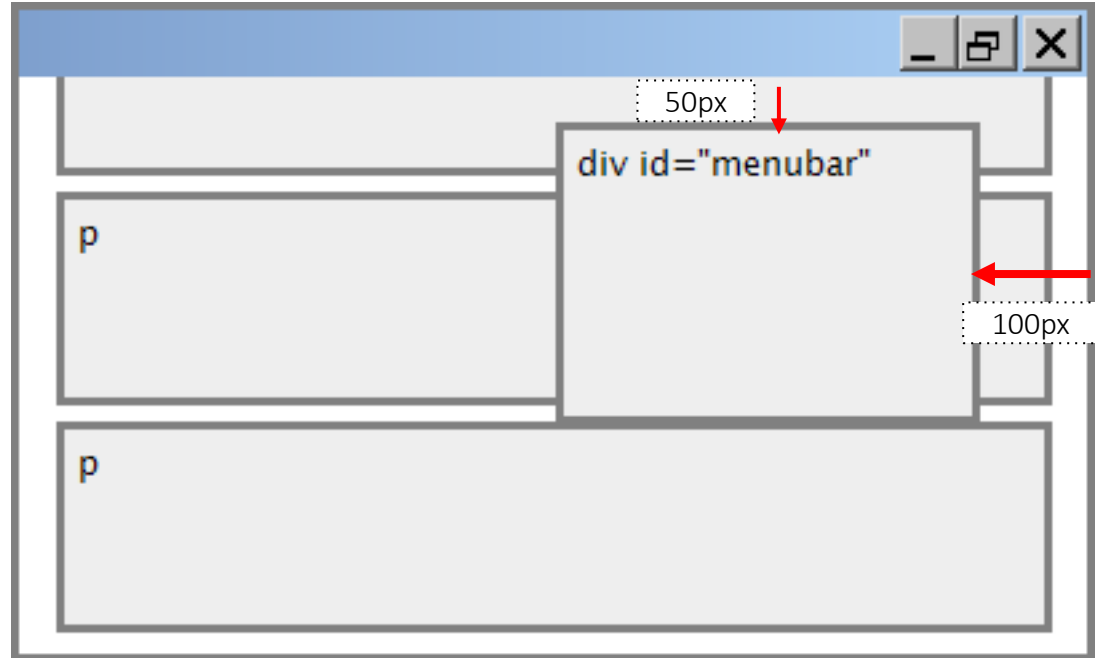
(non succede nulla!)

- **static** è il valore di default: la posizione "naturale" dell'elemento
- Definire lo scostamento con **top** / **left** / **bottom** / **right** senza impostare la proprietà **position** ad un valore diverso da static non serve a nulla

position: fixed

```
#menubar {  
  position: fixed;  
  top: 50px;  
  right: 100px;  
}
```

- Con **position: fixed**, l'offset è relativo all'area di visualizzazione del browser (viewport)
- L'elemento non si muove quando la pagina viene scrollata
- L'elemento è rimosso dal flusso normale



position: fixed

```
#box1 {  
  height: 50px;  
  background-color:  
    rgba(0, 0, 0, 0.5);  
  
  position: fixed;  
  top: 50%;  
  left: 0;  
  right: 0;  
}
```

([JsBin](#))

vernacular English, puppy refers specifically to dogs, while pup may often be used for other mammals such as seals, giraffes, guinea pigs, or even rats.

Development

At first, puppies spend the large majority of their time sleeping and the rest feeding. They instinctively pile together into a heap, and become distressed if separated from physical contact with their littermates, by even a short distance.

Puppies are born with a fully functional sense of smell but can't open their eyes. During their first two weeks, a puppy's senses all develop rapidly. During this stage the nose is the primary sense organ used by puppies to find their mother's teats, and to locate their littermates, if they become separated by a short distance. Puppies open their eyes about nine to eleven days following birth. At first, their retinas are poorly developed and their vision is poor. Puppies are not able to see as well as adult dogs. In addition, puppies' ears remain sealed until about thirteen to seventeen days after birth, after which they respond more actively to sounds. Between two and four weeks old, puppies usually begin to growl, bite, wag their tails, and bark.

Puppies develop very quickly during their first three months, particularly after their eyes and ears open and they are no longer completely dependent on their mother. Their coordination and strength improve, they spar with their littermates, and begin to explore the world outside the nest. They play wrestling, chase, dominance, and tug-of-war games.

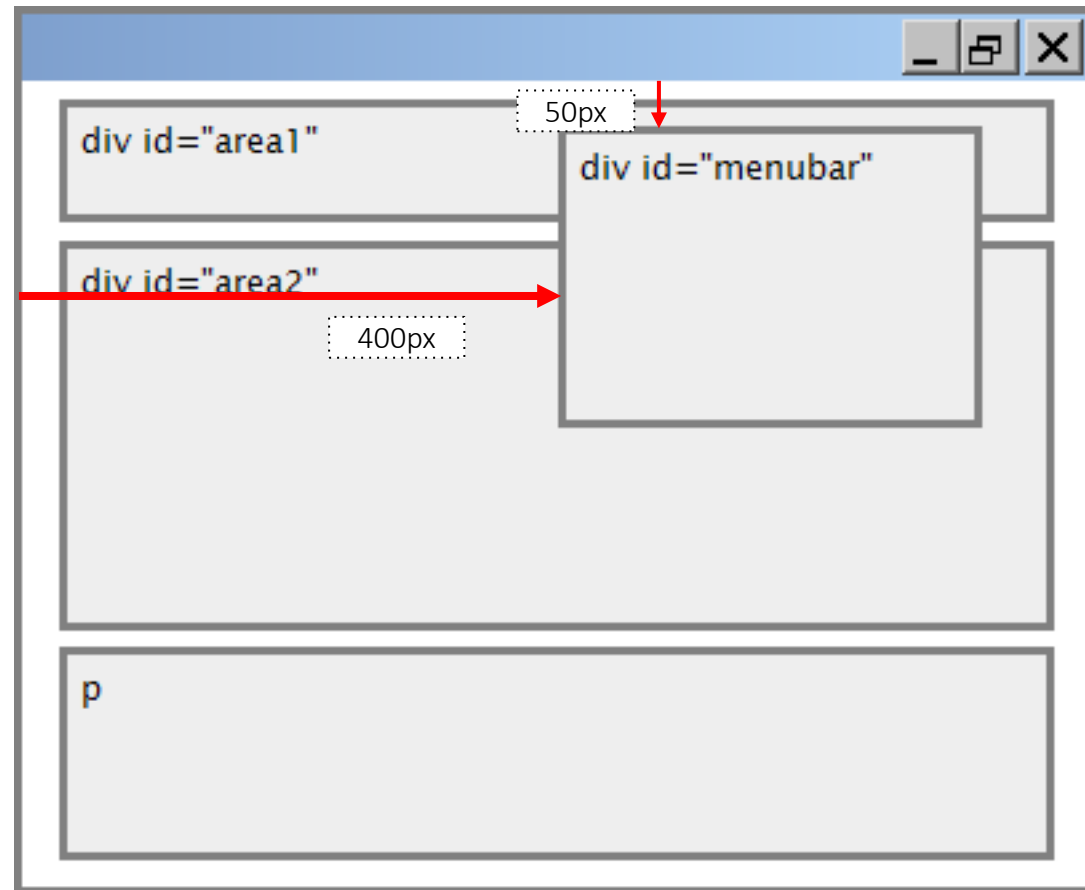
Development

Puppies are highly social animals and spend most of their waking hours interacting with either their mother or littermates. When puppies are socialized with humans, particularly between the ages of eight and twelve weeks, they

position: absolute

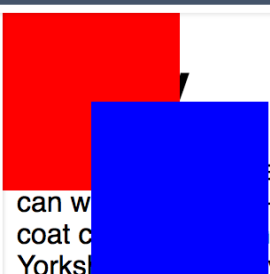
```
#menubar {  
  position: absolute;  
  left: 400px;  
  top: 50px;  
}
```

- Con `position: absolute` l'offset è relativo all'elemento genitore



position: absolute

```
#box1 {  
  height: 100px;  
  width: 100px;  
  background-color: red;  
  
  position: absolute;  
  top: 0;  
  left: 0;  
}  
  
#box2 {  
  height: 100px;  
  width: 100px;  
  background-color: blue;  
  
  position: absolute;  
  top: 50px;  
  left: 50px;  
}
```



the dog. Some puppies can weigh 1–3 lb (0.45–1.36 kg), while larger ones can weigh up to 23 lb (6.8–10.4 kg). All healthy puppies grow quickly after birth. A puppy's coat color can change as the puppy grows older, as is commonly seen in breeds such as the Yorkshire Terrier. In vernacular English, puppy refers specifically to dogs, while pup may often be used for other mammals such as seals, giraffes, guinea pigs, or even rats.

Development

At first, puppies spend the large majority of their time sleeping and the rest feeding. They instinctively pile together into a heap, and become distressed if separated from physical contact with their littermates, by even a short distance.

position: relative

Con `position: relative;` l'offset è definito rispetto a quella che sarebbe la posizione naturale occupata dall'elemento

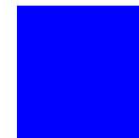
```
#box2 {  
  height: 100px;  
  width: 100px;  
  background-color: blue;  
  
  position: relative;  
  top: 50px;  
  left: 50px;  
}
```

Puppy

A puppy is a juvenile dog. Some puppies can weigh 1–3 lb (0.45–1.36 kg), while larger ones can weigh up to 15–23 lb (6.8–10.4 kg). All healthy puppies grow quickly after birth. A puppy's coat color may change as the puppy grows older, as is commonly seen in breeds such as the Yorkshire Terrier. In vernacular English, puppy refers specifically to dogs, while pup may often be used for other mammals such as seals, giraffes, guinea pigs, or even rats.

Development

At first, puppies spend the large majority of their time sleeping and the rest feeding. They instinctively pile together into a heap, and become distressed if separated from physical contact with their littermates, by even a short distance.



Risorse utili

- <https://leannezhang.medium.com/difference-between-css-position-absolute-versus-relative-35f064384c6#:~:text=position%3A%20relative%20places%20an%20element,changing%20the%20layout%20around%20it>

Altre proprietà CSS utili

La funzione `calc`

La funziona CSS [calc](#) permette di definire valori numeri con espressioni.

Esempi:

```
width: calc(50% - 10px);  
width: calc(100% / 6);
```

([MDN details of calc](#))

Variabili CSS

Le **variabili CSS** consentono di associare un valore ad un identificatore (è una caratteristica recente, verificare su [caniuse](#))

Esempio:

```
:root {  
  --primary-color: hotpink;  
}
```

Definizione della variabile
primary-color

```
h1 {  
  background-color: var(--primary-color);  
}
```

uso della variabile
primary-color

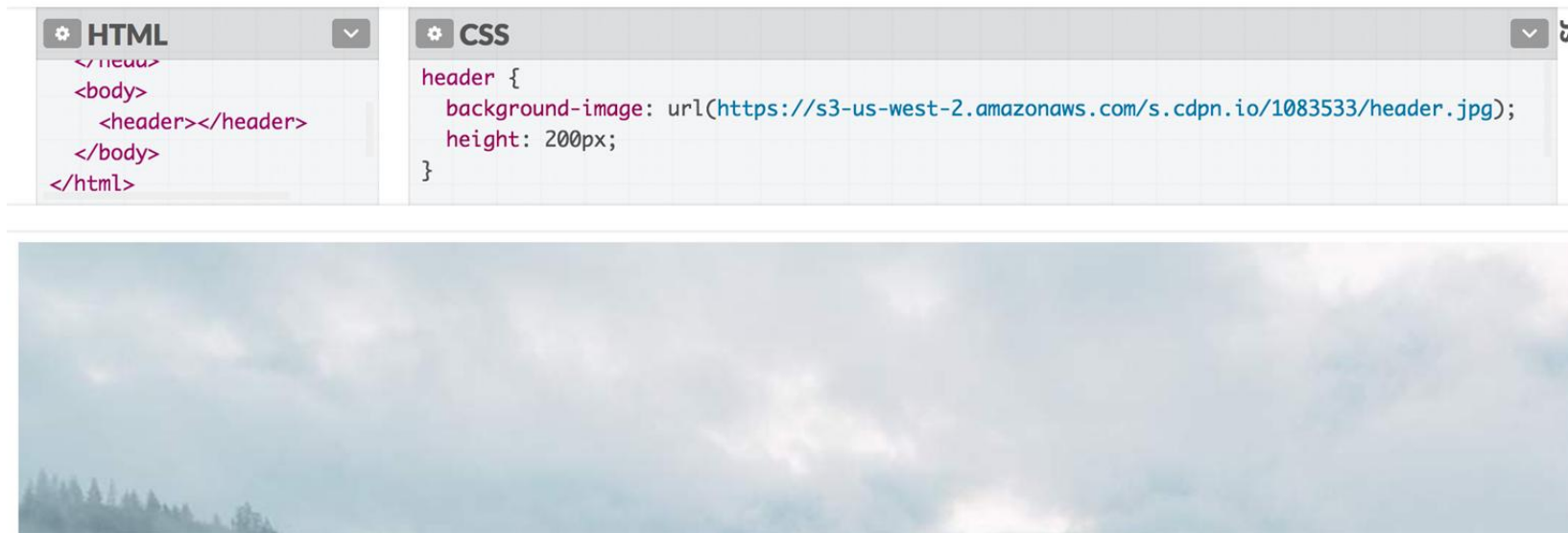
```
.colored {  
  color: var(--primary-color);  
}
```

uso della variabile
primary-color

([MDN details of CSS variables](#))

Proprietà per il background

```
background-image: url(background.png);
```



Proprietà per il background

[Altre proprietà per il background:](#)

```
background-size: cover;  
background-size: contain;  
background-repeat: no-repeat;  
background-position: top;  
background-position: center;
```

Proprietà CSS utili sui font

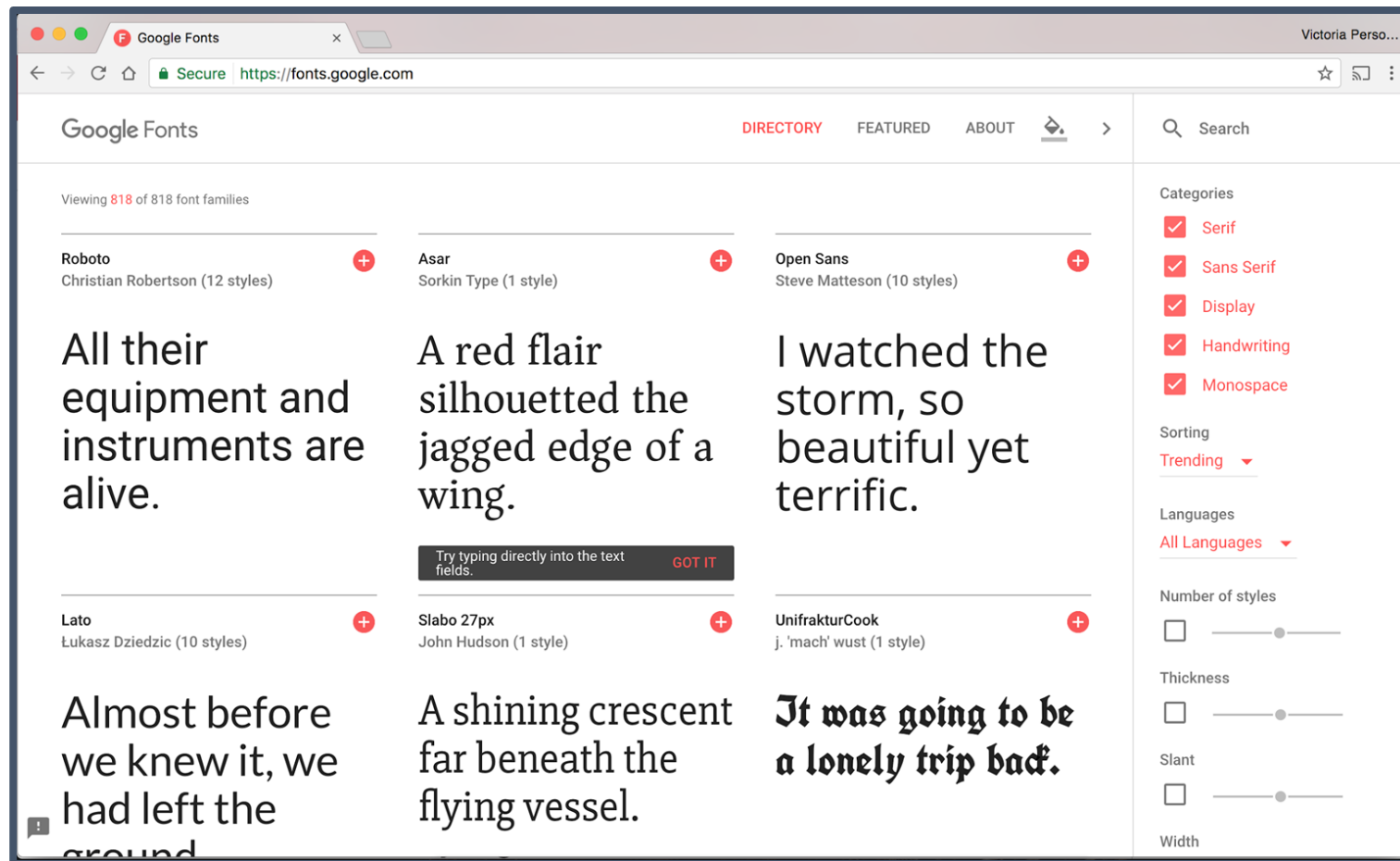
Nome	Descrizione
<code>font-family</code>	Font face (mdn)
<code>color</code>	Font color (and always font color) (mdn)
<code>font-size</code>	Font size (mdn)
<code>line-height</code>	Line height (mdn)
<code>text-align</code>	Alignment of text (mdn)

Altre proprietà CSS utili sui font

Nome	Descrizione
<code>text-decoration</code>	Can set underline , line-through (strikethrough) or none (e.g. to unset underline on hyperlinks) (mdn)
<code>text-transform</code>	Can change font case, i.e. uppercase , lowercase , capitalize , none (mdn)
<code>font-style</code>	Can set to italic or normal (e.g. to unset italic on <code></code>) (mdn)
<code>font-weight</code>	Can set to bold or normal (e.g. to unset bold on h1 - h6) (mdn)
<code>letter-spacing</code>	Controls the space between letters (mdn)

Web Fonts

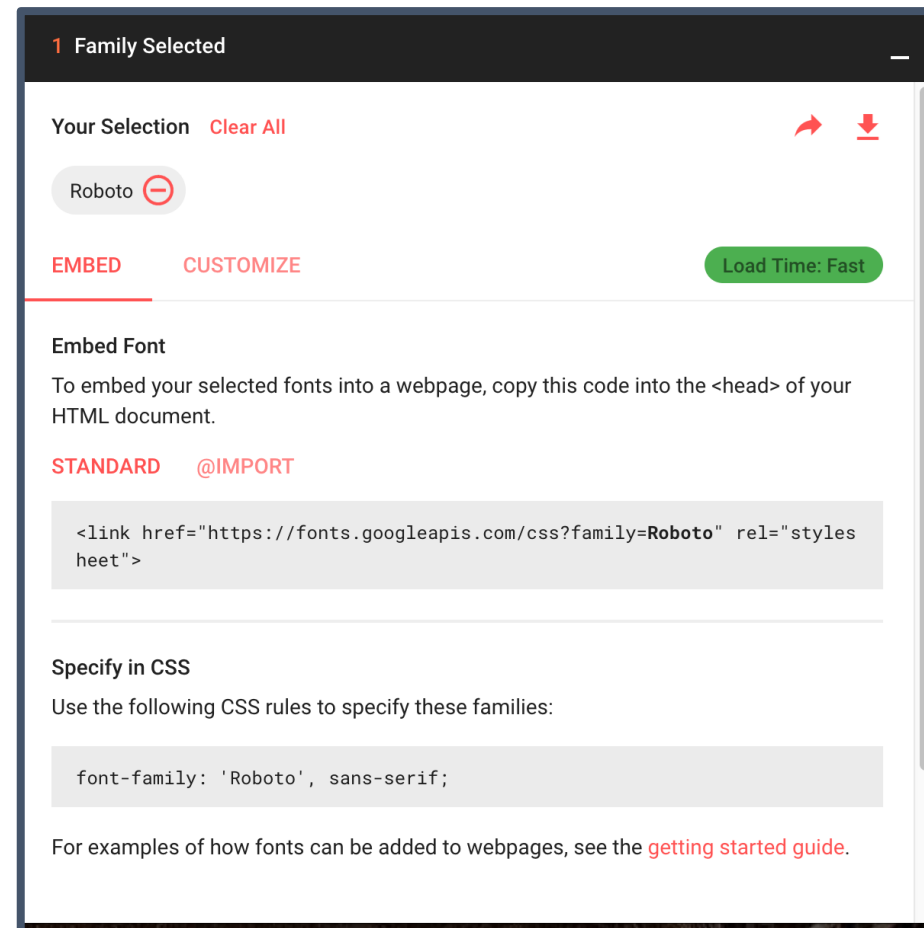
Vedi [Google Fonts](https://fonts.google.com):



Web Fonts

Vedi [Google Fonts](#):

Per usarli, è sufficiente
aggiungere l'elemento
<link> nell'intestazione
html



Mobile web

Responsive web design

Vogliamo scrivere i nostri CSS affinché si vedano bene su schermi di dimensioni molto diverse tra loro:

- 27" desktop monitor
- SmartTV 60"
- Android smartphone
- iPhone X
- iPad

Se non facciamo nulla, anche i browser dei dispositivi mobili presentano la pagina considerando **desktop screen width** (~1000px), quindi "zoommano" fino a far entrare la pagina nello schermo.

Risultato: caratteri piccolissimi!

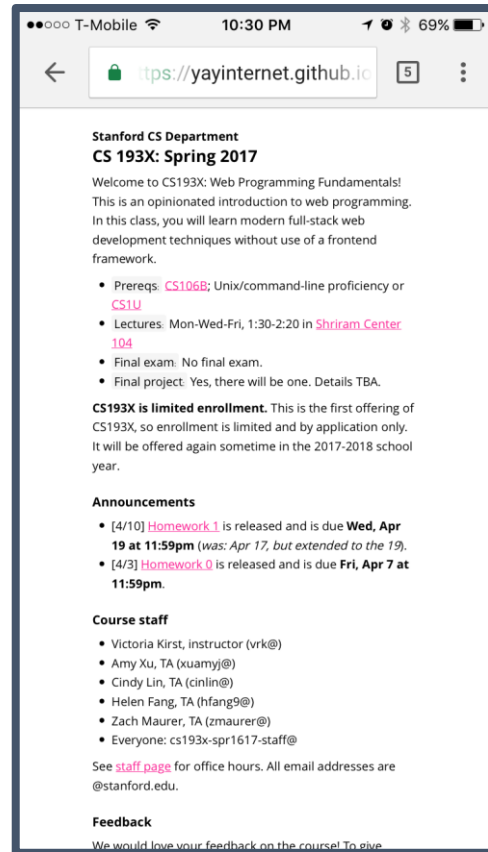
Meta viewport tag

Per prevenire che i browser (degli smartphone) deformino il rendering della pagina zoomando, bisogna usare l'elemento:

```
<meta name="viewport"  
content="width=device-width, initial-scale=1">
```

nell'intestazione <head> della pagina HTML

Meta viewport tag



Senza meta viewport



Con meta viewport

Meta viewport tag

```
<meta name="viewport"  
content="width=device-width, initial-scale=1">
```

- **name=viewport**: "Browser, ti do un'indicazione sulle dimensioni dell'area di visualizzazione (viewport)"
- **width=device-width**: "La larghezza del viewport dovrebbe iniziare sempre alla larghezza del dispositivo"
- **initial-scale=1**: "Inizia con zoom al 100%."

Meta viewport tag

```
<meta name="viewport"  
content="width=device-width, initial-scale=1">
```

Mettiamolo sempre nelle pagine HTML

Aggiustamenti

Il tag meta viewport fa quasi tutto, ma potremmo aver bisogno di fare piccoli aggiustamenti.

Ad esempio ridurre i margini



CSS media query

Una **CSS media query** permette di ridefinire le regole di stile sulla base delle caratteristiche del dispositivo

```
@media (max-width: 500px) {  
  article {  
    margin: 0 2px;  
  }  
}
```

[Vedi esempi più complessi su MDN](#)

Strumenti di sviluppo

Come testare **layout** per dispositivi mobili

- [Chrome device mode](#)
 - (simili strumenti anche in Firefox e Safari)
- [Chrome remote debugging](#)
- [Safari remote debugging](#)

Chrome device mode

Vantaggi:

- Super conveniente
- Accurato, soprattutto con dispositivi Android

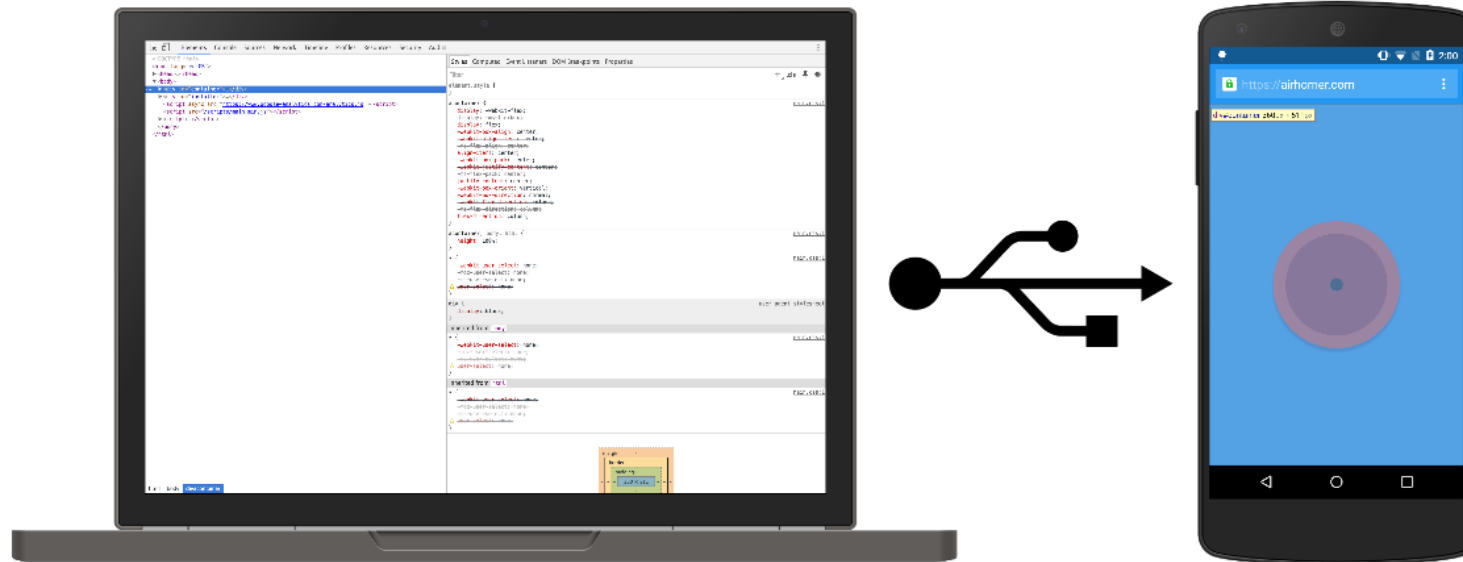
Svantaggi:

- Non sempre accurato con iPhone
- Non simula le prestazioni

Bisognerebbe testare sempre anche su dispositivi reali.

Chrome remote debugging

Con un Android, si possono debuggare le pagine con [Chrome remote debugging](#).



(Quando metterete un'app su un server locale potrete usare anche [port forwarding](#))

Dimensioni relative dei
font: %, **em**, **rem**

Unità relative

Quando è possibile, è meglio usare **unità relative** (ad es. percentuali) al posto di unità assolute (come **px**).

Vantaggi:

- Si adattano più facilmente a diverse dimensioni dello schemo
 - Più intuitive
- 10% / 80% / 10%** vs **122px / 926px / 122px**

Dimensioni relative dei font: percentuali

```
<body>  
  <h1>This is 60px</h1>  
  <p>This is 15px</p>  
</body>
```

```
body {  
  font-size: 30px;  
}  
  
h1 {  
  font-size: 200%;  
}  
  
p {  
  font-size: 50%;  
}
```

This is 60px

This is 15px

Dimensioni relative dei font: percentuali

Percentuale rispetto all'elemento genitore

```
<div>  
  This is 60px  
  <p>This is 45px</p>  
</div>
```

```
body {  
  font-size: 30px;  
}  
  
div {  
  font-size: 200%;  
}  
  
p {  
  font-size: 75%;  
}
```

This is 60px

This is 45px

Dimensioni relative dei font: percentuali

Percentuale rispetto all'elemento genitore

```
<div>  
  This is 60px  
  <p>This is 45px</p>  
</div>
```

```
body {  
  font-size: 30px;  
}  
  
div {  
  font-size: 200%;  
}  
  
p {  
  font-size: 75%;  
}
```

This is 60px

This is 45px

p è 75% del genitore, che è 200% di 30px.

Dimensione di p: $.75 * 2 * 30 = 45\text{px}$

Dimensioni relative dei font: **em**

Di solito si preferisce usare **em** (emphemeral unit): è un fattore moltiplicativo rispetto alla dimensione del font del genitore

Esempi:

- **1em** = font size del genitore
- **2em** = 2 volte font size del genitore
- **.7em** = 0.7 volte font size del genitore

Dimensioni relative dei font: em

```
<body>  
  <h1>This is 60px</h1>  
  <p>This is 15px</p>  
</body>
```

```
body {  
  font-size: 30px;  
}  
  
div {  
  font-size: 2em;  
}  
  
p {  
  font-size: .5em;  
}
```

This is 60px

This is 15px

Dimensioni relative dei font: em

```
<div>  
  This is 60px  
  <p>This is 45px</p>  
</div>
```

```
body {  
  font-size: 30px;  
}  
  
div {  
  font-size: 2em;  
}  
  
p {  
  font-size: .75em;  
}
```

This is 60px

This is 45px

Dimensioni relative dei font: em

```
<div>  
  This is 60px  
  <p>This is 45px</p>  
</div>
```

```
body {  
  font-size: 30px;  
}  
  
div {  
  font-size: 2em;  
}  
  
p {  
  font-size: .75em;  
}
```

This is 60px

This is 45px

La dimensione ereditata da p
è 2em, che è 60px. Quindi
 0.75em è $0.75 \times 60 = 45\text{px}$.

```
<body>  
  This is  
  <h1>  
    <strong>120px</strong>  
  </h1>  
</body>
```

```
body {  
  font-size: 30px;  
}  
  
strong {  
  font-size: 2em;  
}
```

This is

120px

Perchè `<strong>` è 120px e non 60px?

```
<body>
  This is
  <h1>
    <strong>120px</strong>
  </h1>
</body>
```

```
body {
  font-size: 30px;
}

strong {
  font-size: 2em;
}
```

This is

120px

```
h1 {                                     user agent stylesheet
  display: block;
  font-size: 2em;
  -webkit-margin-before: 0.67em;
  -webkit-margin-after: 0.67em;
  -webkit-margin-start: 0px;
  -webkit-margin-end: 0px;
  font-weight: bold;
}
```

Se verifichiamo con Chrome Inspector, ci accorgiamo che la **font-size** per **h1** è **2em**. Quindi $30 * 2 * 2 = 120\text{px}$.

Dimensioni relative dei font: **rem**

Se non vogliamo far riferimento all'elemento genitore, ma alla radice del documento, usiamo rem:

- **rem** rappresenta **font-size** dell'elemento radice
 - **1rem** = root (**html** tag) font size
 - **2rem** = 2 volte root font size

Dimensioni relative dei font: rem

```
<body>
  <div>
    This is 60px
    <p>This is 22.5px</p>
  </div>
</body>
```

```
html {
  font-size: 30px;
}

div {
  font-size: 2rem;
}

p {
  font-size: .75rem;
}
```

This is 60px

This is 22.5px

Dimensioni relative dei font: rem

```
<body>
  <div>
    This is 60px
    <p>This is 22.5px</p>
  </div>
</body>
```

```
html {
  font-size: 30px;
}

div {
  font-size: 2rem;
}

p {
  font-size: .75rem;
}
```

This is 60px

This is 22.5px

font-size impostato
sull'elemento `html`

Dimensioni relative dei font: rem

```
<body>
  <div>
    This is 60px
    <p>This is 22.5px</p>
  </div>
</body>
```

```
html {
  font-size: 30px;
}
```

```
div {
  font-size: 2rem;
}
```

```
p {
  font-size: .75rem;
}
```

This is 60px

This is 22.5px

.75em è calcolato dalla radice, che è 30px. Quindi $30 \cdot .75 = 22.5\text{px}$.

Dimensioni relative dei font: conclusioni

- Preferire **em** and **rem** alle percentuali
 - Più chiaro
- Usare **rem** per evitare dimensioni ereditate
- Oltre a **font-size**, considerare anche **em/rem** for:
 - **line-height**
 - **margin-top**
 - **margin-bottom**

Conclusioni

- Usare sempre il tag **meta viewport tag**
- Usare **@media queries** per aggiungere/aggiustare stili per dispositivi con caratteristiche particolari (ad es. dimensioni dello schermo ridotte)
- Usare **Chrome Device Mode** per simulare il rendering
- Per **height** e **width**, preferire percentuali
- Per i fonts, preferire **em** e **rem**

Per approfondimenti, vedi [responsive web design](#)